

คู่มือทำ Lab 09–13

Authentication, JWT, Cart, File Upload & API Document

ขั้นตอนละเอียดสำหรับโปรเจกต์ KUSHOP ตั้งแต่สร้างตารางในฐานข้อมูล เขียน backend และ frontend ระบบสมาชิก การเข้าสู่ระบบด้วย JWT ใน cookie ตะกร้าสินค้าที่เก็บในฐานข้อมูล การอัปโหลดรูปสมาชิก การ deploy ขึ้น NGINX และการทำเอกสาร API ด้วย Swagger

โค้ดทั้งหมดในเอกสารนี้ดึงมาจากไฟล์จริงในโปรเจกต์ ณ วันที่สร้างเอกสาร
Node.js ^22.18.0 || >=24.12.0 · Express · PostgreSQL · Vue 3 · Pinia

สารบัญ

- ส่วน A** สรุปสิ่งที่ทำไปแล้วใน Lab 09–13
- ส่วน B** สิ่งที่ต้องทำเอง (ทีละขั้น)
- ส่วน C** ติดตั้งบนเครื่องใหม่หลัง clone / pull จาก GitHub
- ส่วน D** ผังโครงสร้างไฟล์ทั้งโปรเจกต์

รายละเอียดแต่ละเรื่อง

1. ภาพรวมระบบ
2. สิ่งที่ต้องเตรียม
3. ฐานข้อมูล: ตาราง members, carts และ cartDtl
4. ตั้งค่าไฟล์ .env
5. Backend สมาชิก: memberController, memberRoute, server.js
6. ทดสอบ Backend ด้วย Postman
7. Frontend: Pinia, authStore, Router
8. Frontend: หน้า Login, Register, PageMember และ MainMenu
9. Lab 11 Backend: ตะกร้าสินค้า (cartController)
10. Lab 11 Frontend: cartStore และหน้าตะกร้า
11. รันและทดสอบทั้งระบบ
12. ปัญหาที่พบบ่อยและวิธีแก้
13. Lab 12: อัปโหลดและแสดงรูปสมาชิก
14. Lab 12: Deploy ด้วย NGINX
15. Lab 13: เอกสาร API ด้วย Swagger (OpenAPI)
16. ส่วนที่ทำเพิ่มนอกแลป (ฟิลเตอร์ ตัวเลือกแบบ custom และการลบข้อมูล)
17. สิ่งที่ต่างจากเอกสารแลป

A สิ่งที่ทำไปแล้วใน Lab 09-13

ส่วนนี้สรุปว่าในโปรเจกต์มีอะไรทำเสร็จแล้วบ้าง **ไม่ต้องเขียนโค้ดเหล่านี้ซ้ำ** โค้ดเต็มอยู่ในหัวข้อ 5, 7-10

A.1 Lab 09 – Authentication

หัวข้อในเอกสารแลป	ทำแล้วที่ไฟล์
<code>postMember</code> : ตรวจสอบข้อมูลครบ, ตรวจสอบอีเมลซ้ำ, hash รหัสผ่านด้วย <code>bcrypt (salt 11)</code> , INSERT ลง <code>members</code>	<code>controllers/memberController.js</code>
<code>loginMember</code> : หาอีเมล แล้วเทียบรหัสด้วย <code>bcrypt.compare</code>	<code>controllers/memberController.js</code>
route <code>POST /members</code> และ <code>POST /members/login</code>	<code>src/router/memberRoute.js</code>
ฟอร์ม Login (ใช้อีเมลเป็น <code>loginName</code>) และฟอร์ม Register ที่ <code>/register</code>	<code>TheLogin.vue</code> , <code>TheRegister.vue</code>

A.2 Lab 10 – Authorization JWT

หัวข้อในเอกสารแลป	ทำแล้วที่ไฟล์
CORS กำหนด origin + <code>credentials: true</code> , เรียก <code>cookieParser()</code> ก่อน router	<code>server.js</code>
<code>SECRET_KEY</code> ใน <code>.env</code>	<code>.env</code>
login สำเร็จสร้าง JWT อายุ 1 ชม. เก็บใน cookie (<code>httpOnly</code> , <code>secure</code> , <code>sameSite: strict</code>), login ผิดลบ cookie	<code>controllers/memberController.js</code>
<code>getMember</code> → <code>GET /members/detail</code> , <code>logoutMember</code> → <code>GET /members/logout</code>	<code>memberController.js</code> , <code>memberRoute.js</code>
<code>axios.defaults.withCredentials = true</code> และติดตั้ง Pinia	<code>src/main.js</code>
หน้า <code>PageMember</code> , เมนูแสดงชื่อสมาชิก + ปุ่มออกจากระบบ, ถ้า login แล้วเข้า <code>/login</code> ให้เปลี่ยนหน้า	<code>PageMember.vue</code> , <code>MainMenu.vue</code> , <code>src/router/index.js</code>

A.3 Lab 11 – Transaction (ตะกร้าสินค้า)

หัวข้อในเอกสารแลป	ฟังก์ชัน / Endpoint	ไฟล์
ตรวจตะกร้าค่าง	<code>chkCart</code> · POST <code>/carts/chkcart</code>	<code>controllers/cartController.js</code>
สร้างตะกร้า (Gen id)	<code>postCart</code> · POST <code>/carts/addcart</code>	<code>controllers/cartController.js</code>
บันทึกสินค้าเข้าตะกร้า	<code>postCartDtl</code> · POST <code>/carts/addcartdtl</code>	<code>controllers/cartController.js</code>
นำสินค้าออก / ลบตะกร้า	<code>delCartDtl</code> , <code>delCart</code> · DELETE <code>/carts/...</code>	<code>controllers/cartController.js</code>
หาค่ารวม	<code>sumCart</code> · GET <code>/carts/sumcart/:id</code>	<code>controllers/cartController.js</code>
แสดงหัว/รายละเอียดตะกร้า	<code>getCart</code> , <code>getCartDtl</code>	<code>controllers/cartController.js</code>
ตะกร้าทั้งหมดของลูกค้า	<code>getCartByCus</code> · POST <code>/carts/getcartbycus</code>	<code>controllers/cartController.js</code>
หน้ารายละเอียดสินค้า + ปุ่มใส่ตะกร้า	route <code>/productshow/:pdId</code>	<code>ProductShow.vue</code>
แสดงตะกร้าบนเมนู	–	<code>CartInfo.vue</code> ใน <code>MainMenu.vue</code>
หน้ารายละเอียดตะกร้า / ตะกร้าทั้งหมด	route <code>/cartshow/:cartId</code> , <code>/cartlist</code>	<code>CartShow.vue</code> , <code>CartList.vue</code>
Store เก็บจำนวนสินค้าในตะกร้า	<code>useCartStore</code>	<code>src/stores/cartStore.js</code>

ทำเพิ่มจากแลป: แก้จำนวนสินค้าในตะกร้า (`setCartDtlQty`) และยืนยันสั่งซื้อ (`cfCart` → ตั้ง `cartCf` เป็น true) ซึ่งแลปมีปุ่มไว้แต่ยังไม่ได้เขียนการทำงาน

ตารางเทียบแบบละเอียดพร้อมเหตุผลอยู่ในหัวข้อ 13

B สิ่งที่ต้องทำเอง (ทีละขั้น)

ยังขาดแค่ฝั่งฐานข้อมูล

โค้ดทั้งหมดพร้อมแล้ว แต่ต้องสร้างตาราง `members`, `carts`, `cartDtl` และตั้งค่าการเชื่อมต่อก่อน ถึงจะสมัครสมาชิก เข้าสู่ระบบ และใช้ตะกร้าได้จริง

ขั้นที่ 1 — เตรียม PostgreSQL (หัวข้อ 2, 3.1)

- ติดตั้งและเปิด PostgreSQL + pgAdmin 4
- ตรวจว่ามีฐานข้อมูล `kushop` ที่มีตาราง `products/brands` จากแลปก่อนหน้า

ขั้นที่ 2 — สร้าง user dev (หัวข้อ 3.2, ข้ามได้ถ้าใช้ postgres)

SQL (รันในฐานะ postgres)

sql

```
CREATE ROLE dev WITH LOGIN PASSWORD '1234';
GRANT CONNECT ON DATABASE kushop TO dev;
GRANT USAGE ON SCHEMA public TO dev;
```

ขั้นที่ 3 — สร้างตาราง members (หัวข้อ 3.3–3.4)

SQL (รันใน Query Tool ของ kushop)

sql

```
CREATE TABLE public.members (
    "memEmail" VARCHAR(100) NOT NULL PRIMARY KEY,
    "memName" VARCHAR(100),
    "dutyId" VARCHAR(100) DEFAULT 'member',
    "memHash" VARCHAR(255)
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.members TO dev;
```

ต้องมีเครื่องหมาย " ครอบชื่อคอลัมน์ ไมอย่างนั้นจะเจอ `column "memEmail" does not exist`

ขั้นที่ 4 — สร้างตาราง carts และ cartDtl (หัวข้อ 3.6, Lab 11)

ถ้าอาจารย์ให้ไฟล์ backup มา ให้ Restore ตาม Lab 11 หน้า 1 หรือรัน SQL นี้แทนก็ได้

SQL (รันใน Query Tool ของ kushop)

sql

```
CREATE TABLE public.carts (
    "cartId" VARCHAR(100) NOT NULL PRIMARY KEY, -- รหัสตะกร้า YYYYMMDD+ลำดับ
    "cusId" VARCHAR(100), -- อีเมลของสมาชิกเจ้าของตะกร้า
    "cartDate" DATE, -- วันที่สร้างตะกร้า
    "cartCf" BOOLEAN DEFAULT false -- ยืนยันสั่งซื้อแล้วหรือยัง
);

CREATE TABLE public."cartDtl" (
    "cartId" VARCHAR(100) NOT NULL,
    "pdId" VARCHAR(100) NOT NULL,
```

```
"qty"    REAL,
"price"  REAL,
PRIMARY KEY ("cartId", "pdId")
);

GRANT SELECT, INSERT, UPDATE, DELETE ON public.carts, public."cartDtl" TO dev;
```

ขั้นที่ 5 — ตรวจสอบตาราง (หัวข้อ 3.5)

```
SQL                                                                    sql

SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_name IN ('members', 'carts', 'cartDtl')
ORDER BY table_name, ordinal_position;
```

ต้องได้ครบ 4 + 4 + 4 คอลัมน์

ขั้นที่ 6 — เติมค่าเชื่อมต่อใน .env (หัวข้อ 4)

.env มี PORT และ SECRET_KEY อยู่แล้ว ห้ามลบ ให้เพิ่มเฉพาะค่าที่ไม่ตรงกับค่าเริ่มต้น (localhost:5432, kushop, postgres / postgres)

```
.env (เพิ่มต่อท้าย)                                                    env

DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=dev
DB_PASSWORD=1234
```

ขั้นที่ 7 — รันระบบ (หัวข้อ 11.1)

```
2 terminal ในโฟลเดอร์โปรเจกต์                                       bash

npm run server    # terminal 1 → http://localhost:3000
npm run dev       # terminal 2 → http://localhost:5173
```

- ถ้า server ตัวเก่าเปิดอยู่ให้ปิด (Ctrl+C) แล้วเปิดใหม่ เพื่อให้อ่าน .env ใหม่
- เปิดเว็บด้วย http://localhost:5173 เท่านั้น ห้ามใช้ 127.0.0.1 เพราะ cookie จะไม่ถูกส่ง

ขั้นที่ 8 — ทดสอบ (หัวข้อ 6 และ 11.2)

ทดสอบ API ด้วย Postman ตามหัวข้อ 6 แล้วทดสอบบนหน้าเว็บตามรายการในหัวข้อ 11.2

ติดปัญหา?

ดูตารางอาการ สาเหตุ และวิธีแก้ในหัวข้อ 12

C ติดตั้งบนเครื่องใหม่ (หลัง clone / pull)

ใช้ตอนย้ายโปรเจกต์ไปเครื่องอื่น GitHub เก็บเฉพาะโค้ด สิ่งที่ไม่ได้ติดไปด้วย มีดังนี้:

สิ่งที่ไม่อยู่ใน git	เหตุผล	ต้องทำอะไร
node_modules/	ไฟล์ใหญ่ ติดตั้งใหม่ได้จาก package.json	npm install (ขั้นที่ C.3)
.env	มีรหัสผ่าน DB และ SECRET_KEY ห้ามขึ้น GitHub	สร้างใหม่ (ขั้นที่ C.4)
ฐานข้อมูล kushop	อยู่ใน PostgreSQL ของแต่ละเครื่อง ไม่ใช่ไฟล์ในโปรเจกต์	ย้ายหรือสร้างใหม่ (ขั้นที่ C.5)

C.1 ติดตั้งโปรแกรมบนเครื่องใหม่

โปรแกรม	ตรวจสอบด้วยคำสั่ง
Git	git --version
Node.js ^22.18.0 >=24.12.0	node -v
PostgreSQL + pgAdmin 4	เปิด pgAdmin แล้วเชื่อมต่อ server ได้

C.2 เอาโค้ดลงเครื่อง

ครั้งแรก

bash

```
git clone https://github.com/Deeptiony/WebT.git
cd WebT
```

มีโปรเจกต์อยู่แล้ว

bash

```
git checkout main
git pull origin main
```

C.3 ติดตั้ง package

Terminal

bash

```
npm install
```

ต้องรันใหม่ทุกครั้งที่ pull แล้ว package.json เปลี่ยน

C.4 สร้างไฟล์ .env

Terminal

bash

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

.env

env

```
PORT=3000

DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=postgres
DB_PASSWORD=รหัสผ่าน-postgres-ของเครื่องนี้

SECRET_KEY=วางค่าที่สุ่มได้ตรงนี้
```

C.5 เตรียมฐานข้อมูลบนเครื่องใหม่

ต้องมีตาราง `products`, `brands`, `pdTypes`, `members`, `carts`, `cartDtl`

เครื่องเดิม: export / เครื่องใหม่: import

bash

```
pg_dump -U postgres -d kushop -f kushop.sql      # เครื่องเดิม
createdb -U postgres kushop                      # เครื่องใหม่
psql -U postgres -d kushop -f kushop.sql         # เครื่องใหม่
```

อย่า commit ไฟล์ `kushop.sql` ขึ้น GitHub เพราะมี hash รหัสผ่านสมาชิกอยู่ ถ้าไม่ย้ายข้อมูล ให้สร้างตารางใหม่ตามส่วน B ขั้นที่ 3-4

C.6 รันและอัปเดตครั้งต่อไป

Terminal

bash

```
npm run server    # terminal 1
npm run dev       # terminal 2

git pull origin main
npm install        # ถ้า package.json เปลี่ยน
# ปิด npm run server (Ctrl+C) แล้วเปิดใหม่ เพื่อให้โค้ด backend ใหม่มีผล
```

C.7 ปัญหาที่เจอบ่อยบนเครื่องใหม่

อาการ	สาเหตุ	วิธีแก้
PowerShell: <code>npm.ps1 cannot be loaded</code>	Windows ปิดการรันสคริปต์	ใช้ Command Prompt หรือรัน <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
<code>Cannot find module 'multer'</code> (หรือ package อื่น)	ยังไม่ได้ <code>npm install</code> หลัง pull	รัน <code>npm install</code>
server ไม่ขึ้น: <code>SECRET_KEY is missing</code>	ไม่มีไฟล์ <code>.env</code> หรือไฟล์ผิดที่	ทำขั้นที่ C.4 ไฟล์ต้องอยู่ข้าง <code>server.js</code>
<code>EADDRINUSE :::3000</code>	มีโปรแกรมอื่นใช้ port 3000	ปิดโปรแกรมนั้น (หน้าเว็บเรียก API ที่ port 3000 ตายตัว)
Vite เปิดที่ 5174 แล้วล็อกอินไม่ได้	port 5173 ถูกใช้อยู่ และ CORS อนุญาตแค่ 5173	ปิดโปรแกรมที่ใช้ 5173 หรือ <code>npm run dev -- --port 5173 --strictPort</code>
<code>git push</code> ได้ 403	Windows จำบัญชี GitHub อื่นไว้	Credential Manager → ลบ <code>git:https://github.com</code> แล้ว push ใหม่

1 ภาพรวมระบบ

Lab 09 (Authentication) สมัครสมาชิกและเข้าสู่ระบบ โดยเก็บรหัสผ่านเป็นค่า hash ของ bcrypt ในตาราง `members`

Lab 10 (Authorization JWT) เข้าสู่ระบบสำเร็จ backend สร้าง JWT ส่งกลับใน cookie แบบ `httpOnly` ทุกคำขอถัดไป browser จะแนบ cookie ไปด้วย backend จึงรู้ว่าใครใช้งานอยู่โดยไม่ต้องเก็บสถานะที่ server

Lab 11 (Transaction) ตะกร้าสินค้าเก็บในฐานข้อมูล 2 ตาราง คือ `carts` (หัวตะกร้า) กับ `cartDtl` (รายการสินค้า) ผูกกับสมาชิกที่ล็อกอินอยู่



Endpoint	หน้าที่
POST <code>/members</code> · POST <code>/members/login</code>	สมัครสมาชิก · เข้าสู่ระบบ (ตั้ง cookie)
GET <code>/members/detail</code> · GET <code>/members/logout</code>	ถอด token · ลบ cookie
POST <code>/carts/chkcart</code> · POST <code>/carts/addcart</code>	ตรวจตะกร้าค่าง · สร้างตะกร้า
POST <code>/carts/addcartdtl</code> · PUT <code>/carts/setcartdtlqty</code>	ใส่สินค้า · แก้จำนวน
DELETE <code>/carts/delcartdtl/:id/:pdId</code> · DELETE <code>/carts/delcart/:id</code>	ลบสินค้า · ลบตะกร้า
PUT <code>/carts/cfcart/:id</code>	ยืนยันสั่งซื้อ
GET <code>/carts/sumcart/:id</code>	ยอดรวมจำนวนและเงิน
GET <code>/carts/getcart/:id</code> · GET <code>/carts/getcartdtl/:id</code>	หัวตะกร้า · รายการในตะกร้า
POST <code>/carts/getcartbycus</code>	ตะกร้าทั้งหมดของสมาชิกที่ล็อกอิน

2 สิ่งที่ต้องเตรียม

โปรแกรม	ใช้ทำอะไร
Node.js ^22.18.0 >=24.12.0	รัน backend และ Vite
PostgreSQL + pgAdmin 4	ฐานข้อมูล kushop
Postman (ไม่บังคับ)	ทดสอบ API ก่อนต่อกับหน้าเว็บ
Google Chrome / Edge	ทดสอบหน้าเว็บ และดู cookie ใน DevTools

```
Terminal bash
npm install
```

ถ้าทำเองจากโปรเจกต์เปล่า ให้ติดตั้งทีละตัวตามแลป:

```
Terminal bash
npm install bcryptjs           # hash รหัสผ่าน (Lab 09)
npm install jsonwebtoken       # สร้าง/ตรวจ JWT (Lab 10)
npm install cookie-parser      # อ่าน cookie ผัง backend (Lab 10)
npm install pinia              # state management ผัง Vue (Lab 10-11)
```

ทำไมใช้ bcryptjs ไม่ใช่ bcrypt

โปรเจกต์นี้มี bcryptjs อยู่แล้ว วิธีเรียกใช้ (hash , compare) เหมือน bcrypt ทุกอย่าง และไม่ต้องคอมไพล์ native module บน Windows

3 ฐานข้อมูล

ทำใน pgAdmin 4 ตามเอกสาร Lab 09 หน้า 2 และ Lab 11 หน้า 1–2 หรือรันคำสั่ง SQL ในส่วน B ขั้นที่ 3–4 แทนก็ได้ ผลเหมือนกัน

3.1 ตรวจสอบว่ามีฐานข้อมูล kushop

- 1 เปิด pgAdmin แล้วเชื่อมต่อ server PostgreSQL
- 2 คูณใน **Databases** ว่ามี `kushop` ถ้ายังไม่มี คลิกขวา **Databases** → **Create** → **Database...**
- 3 คลิกขวาที่ `kushop` → **Query Tool** เพื่อเปิดหน้าต่างรัน SQL

3.2 สร้าง login role dev

เอกสารแลบใช้ user `dev` รหัส `1234` ถ้าจะใช้ `postgres` ก็ข้ามได้ (SQL อยู่ในส่วน B ขั้นที่ 2)

3.3 ตาราง members (Lab 09)

Name	Data type	Length	Not NULL?	Primary key?	Default
<code>memEmail</code>	character varying	100	✓	✓	
<code>memName</code>	character varying	100			
<code>dutyId</code>	character varying	100			<code>'member'</code>
<code>memHash</code>	character varying	255			

3.4 ตาราง carts (Lab 11)

Name	Data type	Length	Not NULL?	Primary key?	ความหมาย
<code>cartId</code>	character varying	100	✓	✓	รหัสตะกร้า
<code>cusId</code>	character varying	100			อีเมลสมาชิกเจ้าของตะกร้า
<code>cartDate</code>	date				วันที่สร้างตะกร้า
<code>cartCf</code>	boolean				ยืนยันสั่งซื้อแล้วหรือยัง (default <code>false</code>)

3.5 ตาราง cartDtl (Lab 11)

Name	Data type	Length	Not NULL?	Primary key?	ความหมาย
cartId	character varying	100	✓	✓	รหัสตะกร้า
pdId	character varying	100	✓	✓	รหัสสินค้า
qty	real				จำนวน
price	real				ราคาต่อหน่วยตอนใส่ตะกร้า

3.6 กำหนดสิทธิ์

คลิกขวาที่ตาราง → **Properties** → **Security** แล้วให้สิทธิ์ INSERT, SELECT, UPDATE, DELETE กับ dev หรือรัน:

SQL

sql

```
GRANT SELECT, INSERT, UPDATE, DELETE ON public.members, public.carts, public."cartDtl" TO dev;
```

ต้องมีเครื่องหมาย " รอบชื่อคอลัมน์

PostgreSQL จะแปลงชื่อที่ไม่มี " เป็นตัวพิมพ์เล็กทั้งหมด (cartId → cartid) แต่ controller query ด้วย "cartId" ถ้าสร้างผิดจะเจอ error column "cartId" does not exist

4 ตั้งค่าไฟล์ .env

สร้างไฟล์ `.env` ที่โฟลเดอร์หลักของโปรเจกต์ (ระดับเดียวกับ `server.js`) ไฟล์นี้อยู่ใน `.gitignore` แล้ว จะไม่ถูก commit

.env

env

```
PORT=3000

# การเชื่อมต่อฐานข้อมูล (ชื่อตัวแปรตาม database.js ของโปรเจกต์นี้)
DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=dev
DB_PASSWORD=1234

# กุญแจลับสำหรับเซ็น JWT
SECRET_KEY=ใส่ค่าสุ่มยาวๆ-ที่นี่
```

server/database.js

js

```
import pg from "pg";
import dotenv from "dotenv";

dotenv.config();

const { Pool } = pg;

const database = new Pool({
  host: process.env.DB_HOST || "localhost",
  port: Number(process.env.DB_PORT || 5432),
  database: process.env.DB_NAME || "kushop",
  user: process.env.DB_USER || "postgres",
  password: process.env.DB_PASSWORD || "postgres"
});

export default database;
```

สุ่มค่า SECRET_KEY

bash

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

ห้ามเปิดเผย SECRET_KEY

ใครรู้ค่านี้อาจสร้าง token ปลอมเป็นสมาชิกคนไหนก็ได้ อย่าใช้ค่าต่างๆ ในงานจริง และอย่า commit ไฟล์ `.env` ถ้า server เปิดโดยไม่มี `SECRET_KEY` จะหยุดทำงานพร้อมข้อความ `SECRET_KEY is missing`

5 Backend สมาชิก

ฟังก์ชัน	แลป	ทำอะไร
postMember	09	ตรวจสอบข้อมูลครบ → ตรวจสอบอีเมลซ้ำ → hash รหัสผ่าน (salt 11) → INSERT
loginMember	09 + 10	หาอีเมล → bcrypt.compare → สำเร็จ: jwt.sign + res.cookie / ล้มเหลว: res.clearCookie
getMember	10	อ่าน req.cookies.token → jwt.verify → ส่งข้อมูลสมาชิก
logoutMember	10	res.clearCookie('token')

controllers/memberController.js

js

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import multer from "multer";
import fs from "fs";
import path from "path";
import database from "../database.js";

// httpOnly keeps the token away from page JavaScript (XSS); strict stops cross-site sends.
const cookieOptions = {
  httpOnly: true,
  secure: true,
  sameSite: 'strict'
};

// The photo is named after the signed-in member, so strip anything that could walk the path
const reserved = /^(default|avatar-.*)$/i;
function photoStem(memEmail) {
  const stem = String(memEmail).replace(/^[A-Za-z0-9@._-]/g, '_');
  // never let a member claim default.jpg or one of the shipped avatars
  return reserved.test(stem) ? '_' + stem : stem;
}

const photoDir = 'img_mem';
const photoExt = (mimetype) => (mimetype === 'image/png' ? 'png' : mimetype === 'image/webp' ? 'webp' : 'jpg');

// the photo keeps its real extension, so look the file up by its stem
function findPhoto(memEmail) {
  const stem = photoStem(memEmail);
  const found = fs.readdirSync(photoDir).find((name) => path.parse(name).name === stem);
  return found ? `/${photoDir}/${found}` : null;
}

// a fresh clone may not have the folder yet, and multer will not create it
fs.mkdirSync("img_mem", { recursive: true });

const photoUpload = multer({
  storage: multer.diskStorage({
    destination: (req, file, callback) => callback(null, 'img_mem'),
```

```

        // the owner comes from the token, never from the form body
        filename: (req, file, callback) => callback(null,
`${photoStem(req.member.memEmail)}.${photoExt(file.mimetype)}`)
    )),
    limits: { fileSize: 5 * 1024 * 1024 },
    fileFilter: (req, file, callback) => {
        callback(null, ['image/jpeg', 'image/png', 'image/webp'].includes(file.mimetype));
    }
}).single('file');

// requireLogin runs first, so req.member is already the verified token
export async function uploadMemberPhoto(req, res) {
    console.log(`POST /members/uploading is requested.`);
    photoUpload(req, res, (err) => {
        if (err) {
            console.log(err.message);
            return res.status(400).json({ error: 'photo.uploadFail' });
        }
        if (!req.file) {
            return res.status(400).json({ error: 'photo.invalid' });
        }
        // a member has one photo: drop the copies left over in other formats
        const stem = photoStem(req.member.memEmail);
        for (const name of fs.readdirSync(photoDir)) {
            if (path.parse(name).name === stem && name !== req.file.filename) {
                fs.unlinkSync(path.join(photoDir, name));
            }
        }
        return res.json({ message: 'photo.uploaded', photo: `/${photoDir}/${req.file.filename}`
    });
    });
}

export async function deleteMemberPhoto(req, res) {
    console.log(`DELETE /members/photo is requested.`);
    try {
        const stem = photoStem(req.member.memEmail);
        let removed = false;
        for (const name of fs.readdirSync(photoDir)) {
            if (path.parse(name).name === stem) {
                fs.unlinkSync(path.join(photoDir, name));
                removed = true;
            }
        }
        if (!removed) return res.status(404).json({ error: 'photo.none' });
        return res.json({ message: 'photo.removed', photo: null });
    } catch (err) {
        console.error(err);
        return res.status(500).json({ error: 'photo.removeFail' });
    }
}

export async function postMember(req, res) {
    console.log(`POST /members is requested.`);
    const bodyData = req.body;
    try {
        // Stored trimmed and lower case so "A@b.com" and "a@b.com" are one account
        const memEmail = String(bodyData.memEmail || '').trim().toLowerCase();
    }
}

```

```

    const memName = String(bodyData.memName || '').trim();
    const password = String(bodyData.password || '');

    if (!memEmail || !memName || !password) {
        return res.json({ message: `ERROR memEmail, memName and password are required.`,
regist: false });
    }
    if (password.length < 6) {
        return res.json({ message: `ERROR password must be at least 6 characters.`, regist:
false });
    }
    if (memEmail.length > 100 || memName.length > 100) {
        return res.json({ message: `ERROR memEmail and memName must be 100 characters or
less.`, regist: false });
    }
    const chkRow = await database.query({
        text: `SELECT * FROM members WHERE LOWER("memEmail") = $1`,
        values: [memEmail]
    });
    if (chkRow.rowCount != 0) {
        return res.json({ message: `ERROR memEmail ${memEmail} is exists.`, regist: false });
    }

    // Store only the bcrypt hash, never the password itself
    const saltround = 11;
    const pwdHash = await bcrypt.hash(password, saltround);
    await database.query({
        text: `INSERT INTO "members" ("memEmail", "memName", "memHash")
        VALUES ($1, $2, $3)`,
        values: [memEmail, memName, pwdHash]
    });
    // Echo back only public fields, not the password
    res.json({
        memEmail,
        memName,
        createDate: new Date(),
        message: "Regist Success",
        regist: true
    });
}
catch (err) {
    return res.json({ message: err.message || `Server error`, regist: false });
}
}

export async function loginMember(req, res) {
    console.log(`POST /members/login is requested.`);
    const bodyData = req.body;
    try {
        const loginName = String(bodyData.loginName || '').trim().toLowerCase();
        if (!loginName || !bodyData.password) {
            return res.json({ message: `Login and Password is required`, login: false });
        }
        const result = await database.query({
            text: `SELECT * FROM members WHERE LOWER("memEmail") = $1`,
            values: [loginName]
        });
        if (result.rowCount == 0) {

```

```

        return res.json({ message: `Login Fail`, login: false });
    }

    const loginOK = await bcrypt.compare(bodyData.password, result.rows[0].memHash);
    if (loginOK) {
        // Token payload: identifying but not secret
        const theuser = {
            memEmail: result.rows[0].memEmail,
            memName: result.rows[0].memName,
            dutyId: result.rows[0].dutyId
        };
        const token = jwt.sign(theuser, process.env.SECRET_KEY, { expiresIn: '1h' });
        res.cookie('token', token, { ...cookieOptions, maxAge: 3600000 });
        res.json({ message: `Login Success`, login: true });
    }
    else {
        res.clearCookie('token', cookieOptions);
        res.json({ message: `Login Fail`, login: false });
    }
}
catch (err) {
    return res.json({ message: err.message || `Server error`, login: false });
}
}

export async function devAdminLogin(req, res) {
    if (process.env.NODE_ENV === 'production' || process.env.ENABLE_DEV_ADMIN_BYPASS !== 'true')
    {
        return res.status(404).json({ error: 'auth.devAdminDisabled' });
    }
    const passcode = typeof req.body?.passcode === 'string' ? req.body.passcode.trim() : '';
    if (!process.env.DEV_ADMIN_PASSCODE || passcode !== process.env.DEV_ADMIN_PASSCODE.trim()) {
        return res.status(401).json({ error: 'auth.devAdminPasscodeInvalid' });
    }
    const adminEmail = String(process.env.DEV_ADMIN_EMAIL || '').trim().toLowerCase();
    if (!adminEmail) {
        return res.status(503).json({ error: 'auth.devAdminAccountUnavailable' });
    }
    try {
        const result = await database.query({
            text: `SELECT "memEmail", "memName", "dutyId" FROM "members" WHERE LOWER("memEmail")
= $1;`,
            values: [adminEmail]
        });
        const admin = result.rows[0];
        if (!admin || admin.dutyId !== 'admin') {
            return res.status(403).json({ error: 'auth.devAdminAccountUnavailable' });
        }
        const token = jwt.sign(admin, process.env.SECRET_KEY, { expiresIn: '1h' });
        res.cookie('token', token, { ...cookieOptions, maxAge: 3600000 });
        return res.json({ login: true });
    } catch (err) {
        console.error(err);
        return res.status(503).json({ error: 'auth.roleCheckFail' });
    }
}

export async function getMember(req, res) {

```

```

console.log(`GET /members/detail is requested.`);
const token = req.cookies.token;
if (!token)
  return res.json({ message: `No member`, login: false });
try {
  // verify fails if the token or its signature was tampered with
  const member = jwt.verify(token, process.env.SECRET_KEY);
  const result = await database.query({
    text: `SELECT "memEmail", "memName", "dutyId" FROM "members" WHERE "memEmail" = $1;`,
    values: [member.memEmail]
  });
  if (!result.rowCount) return res.json({ message: `No member`, login: false });
  const currentMember = result.rows[0];
  return res.json({
    ...currentMember,
    photo: findPhoto(currentMember.memEmail),
    login: true
  });
}
catch (err) {
  console.log(err.message);
  return res.json({ message: `The information was falsified.`, login: false });
}
}

export async function updateMember(req, res) {
  // requireLogin already verified the token cookie and filled req.member

  const { memName, currentPassword, newPassword, confirmPassword } = req.body;
  const nextName = typeof memName === 'string' ? memName.trim() : '';

  if (!nextName) {
    return res.status(400).json({ error: 'profile.nameRequired' });
  }
  if (nextName.length > 100) {
    return res.status(400).json({ error: 'profile.nameTooLong' });
  }
  if (newPassword || currentPassword || confirmPassword) {
    if (!currentPassword || !newPassword || !confirmPassword) {
      return res.status(400).json({ error: 'profile.passwordFieldsRequired' });
    }
    if (newPassword.length < 6) {
      return res.status(400).json({ error: 'profile.passwordTooShort' });
    }
    if (newPassword !== confirmPassword) {
      return res.status(400).json({ error: 'profile.passwordMismatch' });
    }
  }

  try {
    const member = req.member;
    const result = await database.query({
      text: `SELECT "memEmail", "memHash", "dutyId"
      FROM "members" WHERE "memEmail" = $1;`,
      values: [member.memEmail]
    });
    const currentMember = result.rows[0];

```

```

    if (!currentMember) {
      return res.status(404).json({ error: 'profile.notFound' });
    }
    if (newPassword && !(await bcrypt.compare(currentPassword, currentMember.memHash))) {
      return res.status(401).json({ error: 'profile.currentPasswordWrong' });
    }

    const nextHash = newPassword
      ? await bcrypt.hash(newPassword, 11)
      : currentMember.memHash;
    await database.query({
      text: `UPDATE "members"
        SET "memName" = $1, "memHash" = $2
        WHERE "memEmail" = $3;`,
      values: [nextName, nextHash, member.memEmail]
    });

    const nextToken = jwt.sign({
      memEmail: member.memEmail,
      memName: nextName,
      dutyId: currentMember.dutyId
    }, process.env.SECRET_KEY, { expiresIn: '1h' });
    res.cookie('token', nextToken, { ...cookieOptions, maxAge: 3600000 });
    return res.json({
      memEmail: member.memEmail,
      memName: nextName,
      dutyId: currentMember.dutyId,
      photo: findPhoto(member.memEmail),
      login: true
    });
  } catch (err) {
    if (err.name === 'JsonWebTokenError' || err.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'profile.notAuthenticated' });
    }
    console.error(err);
    return res.status(500).json({ error: 'profile.updateFail' });
  }
}

export async function logoutMember(req, res) {
  console.log(`GET /members/logout is requested.`);
  try {
    res.clearCookie('token', cookieOptions);
    res.json({ message: `Logout Success`, login: false });
  }
  catch (err) {
    return res.json({ message: err.message });
  }
}

```

server/routes/memberRoute.js

js

```

import express from "express";
import * as memberC from "../controllers/memberController.js";
import { requireLogin } from "../controllers/authMiddleware.js";

const router = express.Router();

```

```

router.get("/members/detail", memberC.getMember);
router.post("/members/dev-admin", memberC.devAdminLogin);
router.get("/members/logout", memberC.logoutMember);
router.put("/members/profile", requireLogin, memberC.updateMember);
router.post("/members", memberC.postMember);
router.post("/members/login", memberC.loginMember);
router.post("/members/uploadimg", requireLogin, memberC.uploadMemberPhoto);
router.delete("/members/photo", requireLogin, memberC.deleteMemberPhoto);

export default router;

```

server.js

- **CORS** แบบระบุโดเมน พร้อม `credentials: true` เพื่อให้ browser ส่ง cookie เข้ามา port ได้
- `app.use(cookieParser())` ต้องเรียก ก่อน router ไม่อย่างนั้น `req.cookies` จะว่าง
- เรียกใช้ `memberRoutes` และ `cartRoutes`

server/index.js

js

```

import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import cookieParser from "cookie-parser";
import swaggerUI from "swagger-ui-express";
import yaml from "yaml";
import fs from "fs";
import path from "path";
import { fileURLToPath } from "url";

import productRoutes from "./routes/productRoute.js";
import databaseRoutes from "./routes/databaseRoute.js";
import memberRoutes from "./routes/memberRoute.js";
import cartRoutes from "./routes/cartRoute.js";

dotenv.config();

if (!process.env.SECRET_KEY) {
  throw new Error("SECRET_KEY is missing. Add it to .env before starting the API.");
}

const app = express();
const PORT = Number(process.env.PORT || 3000);

// Whitelist the frontend and allow cookies to travel with requests
app.use(cors({
  origin: [
    "http://localhost:5173", "http://127.0.0.1:5173",
    // vite preview and the nginx build (port 80)
    "http://localhost:4173", "http://127.0.0.1:4173",
    "http://localhost", "http://127.0.0.1"
  ],
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: true
}));
app.use(express.json());
app.use("/img_pd", express.static("img_pd"));
// member photos

```

```
app.use("/img_mem", express.static("img_mem"));
// Must run before the routers so req.cookies is filled in
app.use(cookieParser());
app.use(productRoutes);
app.use(databaseRoutes);
app.use(memberRoutes);
app.use(cartRoutes);

// API document (Lab 13): the spec lives next to this file, so it is found
// no matter which folder the server was started from
const here = path.dirname(fileURLToPath(import.meta.url));
const swaggerFile = fs.readFileSync(path.join(here, "swagger.yaml"), "utf-8");
const swaggerDoc = yaml.parse(swaggerFile);
app.use("/api-docs", swaggerUI.serve, swaggerUI.setup(swaggerDoc));

app.listen(PORT, () => {
  console.log(`API server is running at http://localhost:${PORT}`);
});
```

6 ทดสอบ Backend ด้วย Postman

เปิด backend ด้วย `npm run server` แล้วตั้ง Body เป็น raw → JSON

Request	Body	ผลที่ควรได้
POST /members	<code>{"memEmail":"test1@gmail.com","memName":"Mr.First","password":"1234"}</code>	Regist Success . ส่งเข้าได้ ERROR memEmail ... is exists.
POST /members/login	<code>{"loginName":"test1@gmail.com","password":"1234"}</code>	Login Success + cookie token
GET /members/detail	–	ข้อมูลสมาชิก + login: true
GET /members/logout	–	เรียก detail เข้าได้ No member
POST /carts/chkcart	<code>{}</code>	cartExist จริง/เท็จ (ไม่มี cookie ได้ Login is required)
POST /carts/addcart	<code>{}</code>	cartOK: true + เลขตะกร้า
POST /carts/addcartdtl	<code>{"cartId":"...", "pdId":"4"}</code>	cartDtlOK: true
GET /carts/sumcart/:id	–	<code>{ id, qty, money }</code>

ใน pgAdmin คอลัมน์ memHash ต้องขึ้นต้นด้วย \$2 (เป็น hash ไม่ใช่รหัสผ่านจริง) และแถวใน cartDtl ต้องมีราคาตรงกับตาราง products

กรณี	ผลลัพธ์
ไม่มี cookie	<code>{"message":"No member","login":false}</code> / Login is required
แก้ค่า token แม้แต่ตัวเดียว	The information was falsified.
ขอดูตะกร้าของคนอื่น	ได้ผลลัพธ์ว่าง ([])
แก้ตะกร้าที่ยืนยันสั่งซื้อแล้ว	This cart cannot be changed

7 Frontend: Pinia, authStore, Router

7.1 src/main.js

ติดตั้ง Pinia และตั้ง `axios.defaults.withCredentials = true` **ครั้งเดียวที่นี่** ถ้าไม่ตั้ง browser จะไม่ส่ง/รับ cookie ข้าม port

src/main.js

js

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import axios from 'axios'
import App from './App.vue'
import router from './router'
import './assets/main.css'
import './assets/skylearn.css'

// Send the auth cookie with every API request
axios.defaults.withCredentials = true

const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

7.2 src/stores/authStore.js

src/stores/authStore.js

js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'

export const useAuthStore = defineStore('auth', () => {
  const isLogin = ref(false)
  const member = ref(null)
  const checked = ref(false)

  const login = () => {
    isLogin.value = true
  }

  const logout = () => {
    isLogin.value = false
    member.value = null
  }

  // Ask the API to decode the httpOnly token cookie
  const getMember = async () => {
    try {
      const res = await axios.get('http://localhost:3000/members/detail')
      isLogin.value = Boolean(res.data.login)
      member.value = res.data.login ? res.data : null
    } catch (err) {

```

```

        console.log(err.message)
        logout()
    }
    checked.value = true
}

// Clears the httpOnly cookie on the server, then the local state
const memLogout = async () => {
    try {
        await axios.get('http://localhost:3000/members/logout')
    } catch (err) {
        console.log(err.message)
    }
    logout()
}

const updateProfile = async (profile) => {
    const res = await axios.put('http://localhost:3000/members/profile', profile)
    member.value = res.data
    isLogin.value = true
    return res.data
}

return { isLogin, member, checked, login, logout, getMember, memLogout, updateProfile }
})

```

7.3 src/router/index.js

- `meta.requiresAuth` = ต้องล็อกอินก่อน, `meta.guestOnly` = เฉพาะคนที่ยังไม่ล็อกอิน
- `beforeEach` เช็ก cookie ผ่าน `authStore.getMember()` ครั้งแรกที่เปิดเว็บ แล้วตัดสินใจว่าจะให้ไปหน้าไหน
- route ของ Lab 11: `/productshow/:pdId`, `/cartshow/:cartId`, `/cartlist`

src/router/index.js

js

```

import { createRouter, createWebHistory } from 'vue-router';
import TheHome from '../pages/TheHome.vue';
import TheProduct from '../pages/PageProduct.vue';
import TheLogin from '../pages/TheLogin.vue';
import TheRegister from '../pages/TheRegister.vue';
import PageMember from '../pages/PageMember.vue';
import ProductManage from '../pages/ProductManage.vue';
import DatabaseOverview from '../pages/DatabaseOverview.vue';
import ProductShow from '../pages/ProductShow.vue';
import CartShow from '../pages/CartShow.vue';
import CartList from '../pages/CartList.vue';
import { useAuthStore } from '../stores/authStore.js';

const routes = [
  {
    path: '/',
    name: 'Home',
    component: TheHome,
  },
  {
    path: '/product',
    name: 'Product',

```

```

        component: TheProduct,
    },
    {
        path: '/manage',
        name: 'ProductManage',
        component: ProductManage,
        meta: { requiresAuth: true, requiresAdmin: true },
    },
    {
        path: '/pagemember',
        name: 'PageMember',
        component: PageMember,
        meta: { requiresAuth: true },
    },
    {
        path: '/profile',
        redirect: '/pagemember',
    },
    {
        path: '/database',
        name: 'DatabaseOverview',
        component: DatabaseOverview,
        meta: { requiresAuth: true, requiresAdmin: true },
    },
    {
        // The route carries the product id, so the path must name it too
        path: '/productshow/:pdId',
        name: 'ProductShow',
        component: ProductShow,
    },
    {
        path: '/cartshow/:cartId',
        name: 'CartShow',
        component: CartShow,
        meta: { requiresAuth: true },
    },
    {
        path: '/cartlist',
        name: 'CartList',
        component: CartList,
    },
    {
        path: '/cart',
        redirect: '/cartlist',
    },
    {
        path: '/login',
        name: 'Login',
        component: TheLogin,
        meta: { guestOnly: true },
    },
    {
        path: '/register',
        name: 'Register',
        component: TheRegister,
        meta: { guestOnly: true },
    },
];

```

```

const router = createRouter({
  history: createWebHistory(),
  routes,
});

router.beforeEach(async (to) => {
  const authStore = useAuthStore();
  // Check the token cookie once per page load; login/logout keep the store in sync after that
  if (!authStore.checked) await authStore.getMember();

  if (to.meta.requiresAuth && !authStore.isLogin) {
    return { name: 'Login' };
  }

  if (to.meta.requiresAdmin && authStore.member?.dutyId !== 'admin') {
    return { name: 'PageMember' };
  }

  // Already signed in: skip the login/register pages
  if (to.meta.guestOnly && authStore.isLogin) {
    return { name: 'PageMember' };
  }
});

export default router;

```

8 Frontend: หน้าสมาชิก

8.1 TheLogin.vue

src/pages/TheLogin.vue

vue

```
<template>
  <AuthLayout :heading="t('auth.welcome')" :highlight="t('auth.back')">
    <form @submit.prevent="handleSubmit">
      <h2>{{ t('auth.loginTitle') }}</h2>
      <label>{{ t('auth.email') }}<input v-model.trim="loginName" type="email" required
autofocus autocomplete="username"></label>
      <label>{{ t('auth.password') }}<input v-model="password" type="password" required
autocomplete="current-password"></label>
      <button class="auth-submit sk-btn sk-btn-big sk-btn-block" type="submit"
:disabled="busy">{{ t('auth.submitLogin') }} <span></span></button>
    </form>

    <p v-if="message" class="auth-message sk-msg" role="status" :class="{ error: !login }">{{
t(message) }}</p>
    <router-link to="/" class="auth-home-link">{{ t('auth.home') }}</router-link>
    <router-link to="/register" class="auth-switch-link">{{ t('auth.toRegister') }}</router-
link>
    <DevBypass v-if="DevBypass" />
  </AuthLayout>
</template>

<script setup>
import { defineAsyncComponent, ref } from 'vue'
import { useRouter, useRoute } from 'vue-router'
import axios from 'axios'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'
import AuthLayout from '../components/layout/AuthLayout.vue'

const DevBypass = import.meta.env.DEV ? defineAsyncComponent(() =>
import('../components/ui/DevBypass.vue')) : null

const authStore = useAuthStore()
const router = useRouter()
const route = useRoute()
const loginName = ref('')
const password = ref('')
const login = ref(false)
const message = ref('')
const busy = ref(false)

const handleSubmit = async () => {
  busy.value = true
  try {
    const response = await axios.post('http://localhost:3000/members/login', {
      loginName: loginName.value,
      password: password.value
    })
    login.value = response.data.login
    message.value = response.data.message
  }
```

```

        if (login.value) {
            authStore.login()
            await authStore.getMember()
            router.push(typeof route.query.redirect === 'string' ? route.query.redirect :
'/pagemember')
        }
    } catch (err) {
        console.log(err)
        login.value = false
        message.value = 'auth.loginFail'
    } finally {
        busy.value = false
    }
}
</script>

```

8.2 TheRegister.vue

src/pages/TheRegister.vue

vue

```

<template>
  <AuthLayout :heading="t('register.heading')" :highlight="t('register.highlight')">
    <form @submit.prevent="handleSubmit">
      <h2>{{ t('register.title') }}</h2>
      <label>{{ t('auth.email') }}<input v-model.trim="memEmail" type="email" required
autofocus autocomplete="email"></label>
      <label>{{ t('register.name') }}<input v-model.trim="memName" type="text" required
autocomplete="name"></label>
      <label>{{ t('auth.password') }}<input v-model="password" type="password" required
autocomplete="new-password"></label>
      <button class="auth-submit sk-btn sk-btn-big sk-btn-block" type="submit"
:disabled="busy">{{ t('register.submit') }} <span></span></button>
    </form>

    <p v-if="message" class="auth-message sk-msg" role="status" :class="{ error: !regist }">
{{ t(message) }}</p>
    <router-link to="/login" class="auth-switch-link">{{ t('register.toLogin') }}</router-
link>
  </AuthLayout>
</template>

<script setup>
import { ref } from 'vue'
import axios from 'axios'
import { t } from '../i18n.js'
import AuthLayout from '../components/layout/AuthLayout.vue'

const memEmail = ref('')
const memName = ref('')
const password = ref('')
const regist = ref(false)
const message = ref('')
const busy = ref(false)

const handleSubmit = async () => {
  busy.value = true
  try {

```

```

    const response = await axios.post('http://localhost:3000/members', {
      memEmail: memEmail.value,
      memName: memName.value,
      password: password.value
    })
    regist.value = response.data.regist
    message.value = response.data.message
    if (regist.value) password.value = ''
  } catch (err) {
    console.log(err)
    regist.value = false
    message.value = 'register.fail'
  } finally {
    busy.value = false
  }
}
</script>

```

8.3 PageMember.vue

src/pages/PageMember.vue (ไม่รวม CSS)

vue

```

<template>
  <section class="member-page">
    <div class="member-card sk-panel">
      <div class="member-photo">
        
      </div>
      <p class="member-kicker sk-kicker">{{ t('profile.kicker') }}</p>
      <h1>{{ member?.memName }}</h1>

      <dl class="member-details">
        <div><dt>{{ t('auth.email') }}</dt><dd>{{ member?.memEmail }}</dd></div>
        <div><dt>{{ t('member.duty') }}</dt><dd><span class="sk-pill">{{ member?.dutyId
|| '-' }}</span></dd></div>
        <div><dt>{{ t('profile.status') }}</dt><dd class="member-online">{{
t('member.signedIn') }}</dd></div>
      </dl>

      <form class="photo-form" @submit.prevent="uploadFile">
        <p class="photo-title">{{ t('photo.title') }}</p>
        <p class="photo-hint">{{ t('photo.choose') }}</p>
        <label class="sk-file">
          <input type="file" accept="image/jpeg,image/png,image/webp"
@change="onFileChange">
          <span class="sk-file-btn">{{ t('photo.browse') }}</span>
          <span class="sk-file-name" :class="{ picked: fileName }">{{ fileName ||
t('photo.noFile') }}</span>
        </label>
        <button class="sk-btn sk-btn-block" type="submit" :disabled="uploading">
          {{ uploading ? t('photo.uploading') : t('photo.upload') }}
        </button>
        <button v-if="photoPath" class="photo-remove sk-btn sk-btn-coral sk-btn-block"
type="button"
          :disabled="uploading" @click="removePhoto">
          {{ t('photo.remove') }}
        </button>
      </form>
    </div>
  </section>
</template>

```

```

        <div class="photo-presets">
            <span>{{ t('photo.presets') }}</span>
            <button v-for="name in presets" :key="name" class="photo-preset"
type="button"
                :disabled="uploading" :title="t('photo.usePreset')"
@click="usePreset(name)">
                
            </button>
        </div>
        <p v-if="fileMessage" class="photo-message sk-msg" :class="{ error: fileError }"
role="status">{{ t(fileMessage) }}</p>
    </form>

    <button class="profile-edit-button sk-btn sk-btn-block" type="button"
@click="openEditor">
        {{ t('profile.edit') }}
    </button>
    <form v-if="editing" class="profile-form" @submit.prevent="saveProfile">
        <label>
            {{ t('register.name') }}
            <input v-model="profileForm.memName" type="text" maxLength="100" required
autocomplete="name">
        </label>
        <label>
            {{ t('profile.currentPassword') }}
            <input v-model="profileForm.currentPassword" type="password"
autocomplete="current-password">
        </label>
        <label>
            {{ t('profile.newPassword') }}
            <input v-model="profileForm.newPassword" type="password" minLength="6"
autocomplete="new-password">
        </label>
        <label>
            {{ t('profile.confirmPassword') }}
            <input v-model="profileForm.confirmPassword" type="password" minLength="6"
autocomplete="new-password">
        </label>
        <p v-if="profileError" class="profile-message profile-error" role="alert">{{
t(profileError) }}</p>
        <p v-if="profileSaved" class="profile-message profile-success" role="status">{{
t('profile.saved') }}</p>
        <div class="profile-form-actions">
            <button class="profile-save sk-btn sk-btn-block" type="submit"
:disabled="saving">
                {{ saving ? t('profile.saving') : t('profile.save') }}
            </button>
            <button class="profile-cancel sk-btn sk-btn-block" type="button"
@click="editing = false">
                {{ t('profile.cancel') }}
            </button>
        </div>
    </form>
    <router-link to="/product" class="member-shop sk-btn sk-btn-block">{{
t('home.browse') }} </router-link>
    <button class="member-logout sk-btn sk-btn-coral sk-btn-block" type="button"
@click="logout">{{ t('profile.logout') }}</button>
</div>

```

```

    </section>
</template>

<script setup>
import { computed, reactive, ref, watch } from 'vue'
import { useRouter } from 'vue-router'
import axios from 'axios'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'

const authStore = useAuthStore()
const router = useRouter()
const member = computed(() => authStore.member)
const API = 'http://localhost:3000'
const presets = ['avatar-1.jpg', 'avatar-2.jpg', 'avatar-3.jpg']
const photoPath = ref(null)
// bumped after an upload so the browser fetches the new file instead of the cached one
const photoStamp = ref(Date.now())
const file = ref(null)
const fileMessage = ref('')
const fileError = ref(false)
const uploading = ref(false)

// the API reports the stored file (any format), so nothing has to be guessed here
const photoUrl = computed(() => photoPath.value
  ? `${API}${photoPath.value}?t=${photoStamp.value}`
  : `${API}/img_mem/default.jpg`)
watch(() => member.value?.memEmail, () => {
  photoPath.value = member.value?.photo || null
  photoStamp.value = Date.now()
}, { immediate: true })

const sendPhoto = async (blob, name) => {
  uploading.value = true
  fileMessage.value = ''
  fileError.value = false
  try {
    const formData = new FormData()
    formData.append('file', blob, name)
    const response = await axios.post(`${API}/members/uploadimg`, formData, {
      headers: { 'Content-Type': 'multipart/form-data' }
    })
    fileMessage.value = response.data.message || 'photo.uploaded'
    photoPath.value = response.data.photo || photoPath.value
    photoStamp.value = Date.now()
  } catch (err) {
    fileError.value = true
    fileMessage.value = err.response?.data?.error || 'photo.uploadFail'
  } finally {
    uploading.value = false
  }
}

const removePhoto = async () => {
  uploading.value = true
  fileMessage.value = ''
  fileError.value = false
  try {

```

```

        await axios.delete(`${API}/members/photo`)
        photoPath.value = null
        file.value = null
        fileMessage.value = 'photo.removed'
    } catch (err) {
        fileError.value = true
        fileMessage.value = err.response?.data?.error || 'photo.removeFail'
    } finally {
        uploading.value = false
    }
}

const onFileChange = (event) => { file.value = event.target.files?.[0] || null }
const fileName = computed(() => file.value?.name || '')

const uploadFile = async () => {
    if (!file.value) {
        fileError.value = true
        fileMessage.value = 'photo.pickFirst'
        return
    }
    await sendPhoto(file.value, file.value.name)
}

const usePreset = async (name) => {
    try {
        const response = await fetch(`${API}/img_mem/${name}`)
        await sendPhoto(await response.blob(), name)
    } catch (err) {
        console.log(err.message)
        fileError.value = true
        fileMessage.value = 'photo.uploadFail'
    }
}

const editing = ref(false)
const saving = ref(false)
const profileError = ref('')
const profileSaved = ref(false)
const profileForm = reactive({ memName: '', currentPassword: '', newPassword: '',
confirmPassword: '' })

const openEditor = () => {
    profileForm.memName = member.value?.memName || ''
    profileForm.currentPassword = ''
    profileForm.newPassword = ''
    profileForm.confirmPassword = ''
    profileError.value = ''
    profileSaved.value = false
    editing.value = true
}

const saveProfile = async () => {
    profileError.value = ''
    profileSaved.value = false
    saving.value = true
    try {
        await authStore.updateProfile({ ...profileForm })
        profileForm.currentPassword = ''
    }
}

```

```

        profileForm.newPassword = ''
        profileForm.confirmPassword = ''
        profileSaved.value = true
    } catch (error) {
        profileError.value = error.response?.data?.error || 'profile.updateFail'
    } finally {
        saving.value = false
    }
}

const logout = async () => {
    await authStore.memLogout()
    router.push('/login')
}
</script>

```

8.4 MainMenu.vue

src/components/MainMenu.vue – ส่วน script

vue

```

<script setup>
import { computed } from 'vue'
import { useRouter } from 'vue-router'
import { useAuthStore } from '../../stores/authStore.js'
import { t } from '../../i18n.js'
import SkyIcon from '../ui/SkyIcon.vue'
import AppSwitches from './AppSwitches.vue'
import CartInfo from './CartInfo.vue'

const authStore = useAuthStore()
const router = useRouter()
const memName = computed(() => authStore.member?.memName)

const memLogout = async () => {
    await authStore.memLogout()
    router.push('/login')
}

const links = [
    { to: '/', label: 'nav.home', icon: 'home' },
    { to: '/product', label: 'nav.products', icon: 'shop' },
    { to: '/login', label: 'nav.login', icon: 'login', guest: true },
    { to: '/manage', label: 'nav.manage', icon: 'pencil', admin: true },
    { to: '/database', label: 'nav.database', icon: 'database', admin: true },
    { to: '/cartlist', label: 'nav.orders', icon: 'shop', member: true }
]

const visibleLinks = computed(() => links.filter((link) => {
    if (link.guest) return !authStore.isLogin
    if (link.admin) return authStore.isLogin && authStore.member?.dutyId === 'admin'
    if (link.member) return authStore.isLogin
    return true
})))
</script>

```


9 Lab 11 Backend: ตะกร้าสินค้า

ทุก endpoint อ่านเจ้าของตะกร้าจาก token ใน cookie ไม่ใช่จาก body และตรวจว่าตะกร้าเป็นของคนที่ยล็อกอินอยู่จริง

ฟังก์ชัน	ทำอะไร
<code>readMember</code> / <code>ownsCart</code> / <code>canEditCart</code>	อ่านสมาชิกจาก token · ตรวจเจ้าของ · ตรวจว่าตะกร้ายังไม่ถูกยืนยัน
<code>chkCart</code>	หาตะกร้าที่ยังไม่ยืนยัน (<code>cartCf != true</code>) ของสมาชิกคนนี้
<code>postCart</code>	สร้างเลขตะกร้า <code>YYYYMMDD</code> + ลำดับ 4 หลัก แล้ว INSERT (ถ้ามีตะกร้าค้างอยู่แล้วคืนใบเดิม)
<code>postCartDtl</code>	มีสินค้าตัวเดียวกันอยู่แล้วให้บวกจำนวน ถ้ายังไม่มีให้ INSERT ราคาได้จากตาราง <code>products</code>
<code>setCartDtlQty</code> / <code>delCartDtl</code> / <code>delCart</code>	แก้จำนวน · ลบสินค้า · ลบตะกร้าทั้งใบ
<code>cfCart</code>	ยืนยันสั่งซื้อ: ตั้ง <code>cartCf</code> เป็น true (ตะกร้าว่างยืนยันไม่ได้)
<code>sumCart</code> / <code>getCart</code> / <code>getCartDtl</code> / <code>getCartByCus</code>	ยอดรวม · หัวตะกร้า · รายการในตะกร้า · ตะกร้าทั้งหมดของสมาชิก

controllers/cartController.js

js

```
import jwt from "jsonwebtoken";
import database from "../database.js";

// The cart belongs to the signed-in member, so the owner comes from the token
// cookie, never from the request body – a body value could name anyone.
function readMember(req) {
  const token = req.cookies?.token;
  if (!token) return null;
  try {
    return jwt.verify(token, process.env.SECRET_KEY);
  } catch (err) {
    console.log(err.message);
    return null;
  }
}

async function ownsCart(memEmail, cartId) {
  const result = await database.query({
    text: `SELECT "cusId" FROM carts WHERE "cartId" = $1`,
    values: [cartId]
  });
  return result.rows[0]?.cusId === memEmail;
}

// A confirmed cart is an order already placed: it must not change any more
async function canEditCart(memEmail, cartId) {
  const result = await database.query({
    text: `SELECT "cusId", "cartCf" FROM carts WHERE "cartId" = $1`,
```

```

        values: [cartId]
    });
    const cart = result.rows[0];
    return Boolean(cart) && cart.cusId === memEmail && cart.cartCf !== true;
}

export async function chkCart(req, res) {
    const member = readMember(req);
    console.log(`POST CART customer ${member?.memEmail} is requested`);
    if (!member) {
        return res.json({ cartExist: false, error: true, errorMessage: "Login is required" });
    }
    try {
        // An open cart is one that has not been confirmed yet
        const result = await database.query({
            text: `SELECT * FROM carts WHERE "cusId" = $1 AND "cartCf" != true`,
            values: [member.memEmail]
        });
        if (result.rows[0] != null) {
            return res.json({ cartExist: true, cartId: result.rows[0].cartId });
        }
        return res.json({ cartExist: false });
    } catch (err) {
        return res.json({ cartExist: false, error: true, errorMessage: err.message || `Server error` });
    }
}

export async function postCart(req, res) {
    const member = readMember(req);
    console.log(`POST /CART is requested`);
    if (!member) {
        return res.json({ cartOK: false, messageAddCart: "Login is required" });
    }
    try {
        // One open cart per member: a double click must not create a second one
        const open = await database.query({
            text: `SELECT "cartId" FROM carts WHERE "cusId" = $1 AND "cartCf" != true`,
            values: [member.memEmail]
        });
        if (open.rows[0]) {
            return res.json({ cartOK: true, messageAddCart: open.rows[0].cartId });
        }

        // Cart id is YYYYMMDD + a running number for that day
        const now = new Date();
        const year = now.getFullYear();
        const month = String(now.getMonth() + 1).padStart(2, "0");
        const day = String(now.getDate()).padStart(2, "0");
        const currentDate = `${year}${month}${day}`;

        let i = 0;
        let theId = "";
        let existsResult = [];
        do {
            i++;
            theId = `${currentDate}${String(i).padStart(4, "0")}`;
            existsResult = await database.query({

```

```

        text: `SELECT EXISTS (SELECT * FROM carts WHERE "cartId" = $1)`,
        values: [theId]
    });
} while (existsResult.rows[0].exists);

await database.query({
    text: `INSERT INTO carts ("cartId", "cusId", "cartDate") VALUES ($1, $2, $3)`,
    values: [theId, member.memEmail, now]
});
return res.json({ cartOK: true, messageAddCart: theId });
} catch (err) {
    return res.json({ cartOK: false, messageAddCart: err.message || `Server error` });
}
}

export async function postCartDtl(req, res) {
    const member = readMember(req);
    console.log(`POST /CARTDETAIL is requested`);
    if (!member) {
        return res.json({ cartDtlOK: false, messageAddCartDtl: "Login is required" });
    }
    try {
        if (req.body.cartId == null || req.body.pdId == null) {
            return res.json({
                cartDtlOK: false,
                messageAddCartDtl: "CartId && ProductID is required"
            });
        }
        if (!await canEditCart(member.memEmail, req.body.cartId)) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "This cart cannot be changed"
        });
        }

        // The price comes from the products table, never from the request:
        // a body value would let the buyer pick their own price.
        const priceResult = await database.query({
            text: `SELECT "pdPrice" FROM products WHERE "pdId" = $1`,
            values: [req.body.pdId]
        });
        if (priceResult.rowCount === 0) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "Product not found" });
        }
        const price = priceResult.rows[0].pdPrice;

        // Same product already in the cart? add to its quantity instead of a second row
        const pdResult = await database.query({
            text: `SELECT * FROM "cartDtl" ctd WHERE ctd."cartId" = $1 AND ctd."pdId" = $2`,
            values: [req.body.cartId, req.body.pdId]
        });
        const qty = Math.max(1, Math.floor(Number(req.body.qty)) || 1);

        if (pdResult.rowCount == 0) {
            await database.query({
                text: `INSERT INTO "cartDtl" ("cartId", "pdId", "qty", "price") VALUES ($1, $2,
$3, $4)`,
                values: [req.body.cartId, req.body.pdId, qty, price]
            });
        } else {

```

```

        await database.query({
            text: `UPDATE "cartDtl" SET "qty" = $1 WHERE "cartId" = $2 AND "pdId" = $3`,
            values: [pdResult.rows[0].qty + qty, req.body.cartId, req.body.pdId]
        });
    }
    return res.json({ cartDtlOK: true, messageAddCartDtl: req.body.cartId });
} catch (err) {
    return res.json({ cartDtlOK: false, messageAddCartDtl: err.message || `Server error` });
}
}

export async function setCartDtlQty(req, res) {
    const member = readMember(req);
    console.log(`PUT /CARTDETAIL is requested`);
    if (!member) {
        return res.json({ cartDtlOK: false, messageAddCartDtl: "Login is required" });
    }
    try {
        const qty = Math.floor(Number(req.body.qty))
        if (req.body.cartId == null || req.body.pdId == null || !(qty > 0)) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "CartId && ProductID && Qty is required" });
        }
        if (!await canEditCart(member.memEmail, req.body.cartId)) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "This cart cannot be changed" });
        }
        await database.query({
            text: `UPDATE "cartDtl" SET "qty" = $1 WHERE "cartId" = $2 AND "pdId" = $3`,
            values: [qty, req.body.cartId, req.body.pdId]
        });
        return res.json({ cartDtlOK: true, messageAddCartDtl: req.body.cartId });
    } catch (err) {
        return res.json({ cartDtlOK: false, messageAddCartDtl: err.message || `Server error` });
    }
}

export async function delCartDtl(req, res) {
    const member = readMember(req);
    console.log(`DELETE /CARTDETAIL ${req.params.id}/${req.params.pdId} is requested`);
    if (!member) {
        return res.json({ delOK: false, message: "Login is required" });
    }
    try {
        if (!await canEditCart(member.memEmail, req.params.id)) {
            return res.json({ delOK: false, message: "This cart cannot be changed" });
        }
        await database.query({
            text: `DELETE FROM "cartDtl" WHERE "cartId" = $1 AND "pdId" = $2`,
            values: [req.params.id, req.params.pdId]
        });
        return res.json({ delOK: true, message: `Delete detail success` });
    } catch (err) {
        return res.json({ delOK: false, message: err.message || `Server error` });
    }
}

export async function delCart(req, res) {

```

```

const member = readMember(req);
console.log(`DELETE /CART ${req.params.id} is requested`);
if (!member) {
    return res.json({ delOK: false, message: "Login is required" });
}
try {
    if (!await canEditCart(member.memEmail, req.params.id)) {
        return res.json({ delOK: false, message: "This cart cannot be changed" });
    }
    // Details first: they reference the cart
    await database.query({
        text: `DELETE FROM "cartDtl" WHERE "cartId" = $1`,
        values: [req.params.id]
    });
    await database.query({
        text: `DELETE FROM carts WHERE "cartId" = $1`,
        values: [req.params.id]
    });
    return res.json({ delOK: true, message: `Delete cart success` });
} catch (err) {
    return res.json({ delOK: false, message: err.message || `Server error` });
}
}

export async function cfCart(req, res) {
    const member = readMember(req);
    console.log(`PUT /CART CONFIRM ${req.params.id} is requested`);
    if (!member) {
        return res.json({ cfOK: false, message: "Login is required" });
    }
    try {
        if (!await canEditCart(member.memEmail, req.params.id)) {
            return res.json({ cfOK: false, message: "This cart cannot be changed" });
        }
        // An empty cart cannot be ordered
        const countResult = await database.query({
            text: `SELECT COUNT(*) AS rows FROM "cartDtl" WHERE "cartId" = $1`,
            values: [req.params.id]
        });
        if (Number(countResult.rows[0].rows) === 0) {
            return res.json({ cfOK: false, message: `Cart is empty` });
        }
        await database.query({
            text: `UPDATE carts SET "cartCf" = true WHERE "cartId" = $1`,
            values: [req.params.id]
        });
        return res.json({ cfOK: true, message: `Confirm success` });
    } catch (err) {
        return res.json({ cfOK: false, message: err.message || `Server error` });
    }
}

export async function sumCart(req, res) {
    console.log(`GET SumCart ${req.params.id} is requested`);
    const member = readMember(req);
    if (!member) return res.json({ id: req.params.id, qty: 0, money: 0 });
    try {
        if (!await ownsCart(member.memEmail, req.params.id)) {

```

```

        return res.json({ id: req.params.id, qty: 0, money: 0 });
    }
    const result = await database.query({
        text: `SELECT SUM(qty) AS qty, SUM(qty * price) AS money
            FROM "cartDtl" ctd WHERE ctd."cartId" = $1`,
        values: [req.params.id]
    });
    return res.json({
        id: req.params.id,
        qty: result.rows[0].qty,
        money: result.rows[0].money
    });
} catch (err) {
    return res.json({ error: err.message || `Server error` });
}
}

export async function getCart(req, res) {
    console.log(`GET Cart ${req.params.id} is requested`);
    const member = readMember(req);
    if (!member) return res.json([]);
    try {
        const result = await database.query({
            text: `SELECT ct.*, SUM(ctd.qty) AS sqty, SUM(ctd.price * ctd.qty) AS sprice
                FROM carts ct LEFT JOIN "cartDtl" ctd ON ct."cartId" = ctd."cartId"
                WHERE ct."cartId" = $1 AND ct."cusId" = $2
                GROUP BY ct."cartId"`,
            values: [req.params.id, member.memEmail]
        });
        return res.json(result.rows);
    } catch (err) {
        return res.json({ error: err.message || `Server error` });
    }
}

export async function getCartDtl(req, res) {
    console.log(`GET CartDtl ${req.params.id} is requested`);
    const member = readMember(req);
    if (!member) return res.json([]);
    try {
        if (!await ownsCart(member.memEmail, req.params.id)) return res.json([]);
        const result = await database.query({
            text: `SELECT ROW_NUMBER() OVER (ORDER BY ctd."pdId") AS row_number,
                ctd."pdId", pd."pdName", ctd.qty, ctd.price
                FROM "cartDtl" ctd LEFT JOIN "products" pd ON ctd."pdId" = pd."pdId"
                WHERE ctd."cartId" = $1
                ORDER BY ctd."pdId"`,
            values: [req.params.id]
        });
        return res.json(result.rows);
    } catch (err) {
        return res.json({ error: err.message || `Server error` });
    }
}

export async function getCartByCus(req, res) {
    console.log(`POST Cart By Customer is requested`);
    const member = readMember(req);

```

```

if (!member) return res.json([]);
try {
  const result = await database.query({
    text: `SELECT ROW_NUMBER() OVER (ORDER BY ct."cartId" DESC) AS row_number,
          ct.*, SUM(ctd.qty) AS sqty, SUM(ctd.price * ctd.qty) AS sprice
          FROM carts ct LEFT JOIN "cartDtl" ctd ON ct."cartId" = ctd."cartId"
          WHERE ct."cusId" = $1
          GROUP BY ct."cartId"
          ORDER BY ct."cartId" DESC`,
    values: [member.memEmail]
  });
  return res.json(result.rows);
} catch (err) {
  return res.json({ error: err.message || `Server error` });
}
}

```

server/routes/cartRoute.js

js

```

import express from "express";
import * as cartC from "../controllers/cartController.js";

const router = express.Router();

router.post("/carts/chkcart", cartC.chkCart);
router.post("/carts/addcart", cartC.postCart);
router.post("/carts/addcartdtl", cartC.postCartDtl);
router.put("/carts/setcartdtlqty", cartC.setCartDtlQty);
router.get("/carts/sumcart/:id", cartC.sumCart);
router.get("/carts/getcart/:id", cartC.getCart);
router.get("/carts/getcartdtl/:id", cartC.getCartDtl);
router.post("/carts/getcartbycus", cartC.getCartByCus);
router.put("/carts/cfcart/:id", cartC.cfCart);
router.delete("/carts/delcartdtl/:id/:pdId", cartC.delCartDtl);
router.delete("/carts/delcart/:id", cartC.delCart);

export default router;

```

10 Lab 11 Frontend: ตะกร้าสินค้า

10.1 src/stores/cartStore.js

Store กลางของตะกร้า เก็บเลขตะกร้า จำนวนชิ้น และยอดเงิน ให้ทุก component ใช้ร่วมกัน

src/stores/cartStore.js

js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'
import { useAuthStore } from '../authStore.js'

const API = 'http://localhost:3000'
const GUEST_CART_KEY = 'kushop-guest-cart'

export const useCartStore = defineStore('cart', () => {
  const cartId = ref(null)
  const theQty = ref(0)
  const money = ref(0)
  const guestItems = ref([])
  const authStore = useAuthStore()

  const loadGuestCart = () => {
    try {
      const saved = JSON.parse(localStorage.getItem(GUEST_CART_KEY) || '[]')
      guestItems.value = Array.isArray(saved) ? saved : []
    } catch {
      guestItems.value = []
    }
  }

  const saveGuestCart = () => {
    localStorage.setItem(GUEST_CART_KEY, JSON.stringify(guestItems.value))
    theQty.value = guestItems.value.reduce((sum, item) => sum + Number(item.qty), 0)
    money.value = guestItems.value.reduce((sum, item) => sum + Number(item.price) *
Number(item.qty), 0)
  }

  // Which cart is still open for this member (not confirmed yet)
  const chkCart = async () => {
    const res = await axios.post(`${API}/carts/chkcart`, {})
    cartId.value = res.data.cartExist ? res.data.cartId : null
    return cartId.value
  }

  const addCart = async () => {
    const res = await axios.post(`${API}/carts/addcart`, {})
    if (!res.data.cartOK) throw new Error(res.data.messageAddCart)
    cartId.value = res.data.messageAddCart
    return cartId.value
  }

  const sumCart = async (id = cartId.value) => {
    if (!id) {
      theQty.value = 0
    }
  }
})
```

```

        money.value = 0
        return
    }
    const res = await axios.get(`${API}/carts/sumcart/${id}`)
    theQty.value = Number(res.data.qty) || 0
    money.value = Number(res.data.money) || 0
}

// Read the open cart and its totals; used on load and after every change
const refresh = async () => {
    loadGuestCart()
    if (!authStore.isLogin) {
        saveGuestCart()
        return
    }
    try {
        await chkCart()
        for (const item of guestItems.value) await addServerProduct(item)
        if (guestItems.value.length) {
            guestItems.value = []
            saveGuestCart()
        }
        await sumCart()
    } catch (err) {
        console.log(err.message)
        reset()
    }
}

const addServerProduct = async (product, qty = 1) => {
    if (!cartId.value) await chkCart()
    if (!cartId.value) await addCart()

    const res = await axios.post(`${API}/carts/addcartdtl`, {
        cartId: cartId.value,
        pdId: product.pdId,
        pdPrice: product.pdPrice ?? product.price,
        qty
    })
    if (!res.data.cartDtlOK) throw new Error(res.data.messageAddCartDtl)
}

// Creates the cart on first use, then adds the product to it
const addProduct = async (product, qty = 1) => {
    if (!authStore.isLogin) {
        loadGuestCart()
        const existing = guestItems.value.find(item => item.pdId === product.pdId)
        if (existing) {
            existing.qty += qty
        } else {
            guestItems.value.push({
                pdId: product.pdId,
                pdName: product.pdName,
                price: Number(product.pdPrice ?? product.price) || 0,
                qty,
                logosrc: product.logosrc || '',
                brand: product.brand || null
            })
        }
    }
}

```

```

        }
        saveGuestCart()
        return
    }

    await addServerProduct(product, qty)
    await sumCart()
}

const removeGuestProduct = (pdId) => {
    loadGuestCart()
    guestItems.value = guestItems.value.filter(item => item.pdId !== pdId)
    saveGuestCart()
}

const adjustGuestQty = (pdId, qty) => {
    loadGuestCart()
    const item = guestItems.value.find(entry => entry.pdId === pdId)
    if (!item) return
    if (qty <= 0) {
        removeGuestProduct(pdId)
        return
    }
    item.qty = qty
    saveGuestCart()
}

const setQty = async (id, pdId, qty) => {
    const res = await axios.put(`${API}/carts/setcartdtlqty`, { cartId: id, pdId, qty })
    if (!res.data.cartDtlOK) throw new Error(res.data.messageAddCartDtl)
    await sumCart()
}

const removeProduct = async (id, pdId) => {
    const res = await axios.delete(`${API}/carts/delcartdtl/${id}/${pdId}`)
    if (!res.data.delOK) throw new Error(res.data.message)
    await refresh()
}

const deleteCart = async (id) => {
    const res = await axios.delete(`${API}/carts/delcart/${id}`)
    if (!res.data.delOK) throw new Error(res.data.message)
    await refresh()
}

// Confirming closes the cart, so the next add starts a new one
const confirmCart = async (id) => {
    const res = await axios.put(`${API}/carts/cfcart/${id}`)
    if (!res.data.cfOK) throw new Error(res.data.message)
    await refresh()
}

const reset = () => {
    cartId.value = null
    theQty.value = 0
    money.value = 0
}

```

```
loadGuestCart()
saveGuestCart()
```

```
return { cartId, theQty, money, guestItems, chkCart, addCart, sumCart, refresh, addProduct,
removeGuestProduct, adjustGuestQty, setQty, removeProduct, deleteCart, confirmCart, reset }
})
```

10.2 ProductShow.vue (หน้ารายละเอียดสินค้า)

อ่าน `route.params.pdId` แล้วเรียก `GET /products/:id` ปุ่มใส่ตะกร้าจะตรวจการล็อกอินก่อน แล้วเรียก `cartStore.addProduct()` ซึ่งจะตรวจตะกร้าค้าง สร้างตะกร้าถ้ายังไม่มี และบันทึกสินค้าให้ตามลำดับ

src/pages/ProductShow.vue (ไม่รวม CSS)

vue

```
<template>
  <section class="detail-page sk-page">
    <router-link to="/product" class="detail-back">{{ t('detail.back') }}</router-link>

    <p v-if="loading" class="detail-state" aria-busy="true">{{ t('db.loading') }}</p>
    <div v-else-if="!product" class="detail-state sk-empty">
      <h2>{{ t('detail.notFound') }}</h2>
      <router-link to="/product" class="sk-btn">{{ t('shop.showAll') }}</router-link>
    </div>

    <article v-else class="detail-layout">
      <div class="detail-image sk-card">
        
      </div>

      <div class="detail-info">
        <p class="sk-kicker">{{ t('detail.kicker') }}</p>
        <h1>{{ product.pdName }}</h1>
        <p class="detail-price sk-mono">${{ Number(product.pdPrice).toFixed(2) }}</p>

        <dl class="detail-table">
          <div><dt>{{ t('detail.id') }}</dt><dd class="sk-mono">{{ product.pdId }}</dd>
          <div><dt>{{ t('detail.brand') }}</dt><dd>{{ product.brand?.brandName ||
t('product.noBrand') }}</dd></div>
          <div v-if="product.pdt?.pdTypeName"><dt>{{ t('detail.type') }}</dt><dd>{{
product.pdt.pdTypeName }}</dd></div>
          <div><dt>{{ t('detail.remark') }}</dt><dd>{{ product.pdRemark || '-' }}</dd>
        </div>

        <div class="detail-quantity" role="group" :aria-label="t('cartdb.qty')">
          <span>{{ t('cartdb.qty') }}</span>
          <div class="quantity-control">
            <button type="button" :disabled="quantity <= 1 || busy" :aria-
label="t('cart.decrease', { name: product.pdName })" @click="quantity--"></button>
            <output class="sk-mono" aria-live="polite">{{ quantity }}</output>
            <button type="button" :disabled="busy" :aria-label="t('cart.increase', {
name: product.pdName })" @click="quantity++"></button>
          </div>
        </div>

        <div class="detail-actions">
```

```

        <button class="sk-btn sk-btn-big" type="button" :disabled="busy"
@click="addToCart">
            <SkyIcon :name="added ? 'check' : 'cart'" :size="22" />
            {{ t(added ? 'product.added' : 'product.add') }}
        </button>
        <router-link v-if="authStore.member?.dutyId === 'admin'" to="/manage"
class="sk-btn sk-btn-big sk-btn-ghost">
            <SkyIcon name="pencil" :size="22" />
            {{ t('manage.edit') }}
        </router-link>
    </div>

    <p v-if="message" class="sk-msg error" role="status">{{ t(message) }}</p>
    <p v-else-if="added" class="sk-msg" role="status">{{ t('product.addedLive', {
name: product.pdName }) }}</p>
    </div>
</article>
</section>
</template>

<script setup>
import { computed, onMounted, ref } from 'vue'
import { useRoute } from 'vue-router'
import axios from 'axios'
import { useCartStore } from '../stores/cartStore.js'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'
import SkyIcon from '../components/ui/SkyIcon.vue'

const route = useRoute()
const cartStore = useCartStore()
const authStore = useAuthStore()

const product = ref(null)
const loading = ref(true)
const busy = ref(false)
const added = ref(false)
const message = ref('')
const quantity = ref(1)

const imageSrc = computed(() => product.value?.logosrc
? `http://localhost:3000${product.value.logosrc}`
: `http://localhost:3000/products/${product.value?.pdId}/image`)

onMounted(async () => {
    try {
        // Backend answers with an array of matching rows
        const res = await axios.get(`http://localhost:3000/products/${route.params.pdId}`)
        product.value = res.data[0] || null
    } catch (err) {
        console.log(err.message)
    } finally {
        loading.value = false
    }
})

const addToCart = async () => {
    busy.value = true

```

```

    try {
      await cartStore.addProduct(product.value, quantity.value)
      added.value = true
      message.value = ''
    } catch (err) {
      console.log(err.message)
      added.value = false
      message.value = 'product.addFail'
    } finally {
      busy.value = false
    }
  }
}
</script>

```

10.3 CartInfo.vue (ปุ่มตะกร้าบนเมนู)

src/components/layout/CartInfo.vue (ไม่รวม CSS)

vue

```

<template>
  <router-link :to="target" class="cart-info sky-cart"
    :aria-label="t('nav.cartAria', { n: cartStore.theQty })">
    <SkyIcon name="cart" :size="24" />
    <span class="cart-info-label">{{ t('nav.cart') }}</span>
    <span class="cart-info-badge sk-mono">{{ cartStore.theQty }}</span>
    <span v-if="cartStore.money > 0" class="cart-info-money sk-mono">${{
cartStore.money.toFixed(2) }}</span>
  </router-link>
</template>

<script setup>
import { computed, onMounted } from 'vue'
import { useCartStore } from '../../stores/cartStore.js'
import { t } from '../../i18n.js'
import SkyIcon from '../../ui/SkyIcon.vue'

const cartStore = useCartStore()

// No open cart yet? send them to the order history instead of a dead link
const target = computed(() => cartStore.cartId ? `/cartshow/${cartStore.cartId}` : '/cartlist')

onMounted(() => {
  cartStore.refresh()
})
</script>

```

10.4 CartShow.vue (รายละเอียดตะกร้า)

อ่านหัวตะกร้ากับรายการสินค้าพร้อมกัน ถ้าตะกร้าไม่ใช่ของผู้ใช้คนนี้ API จะคืนค่าว่างและหน้าจะขึ้นว่าไม่มีสิทธิ์ดู ตะกร้าที่ยืนยันแล้วจะซ่อนปุ่มแก้ไขทั้งหมด

src/pages/CartShow.vue – ส่วน script

vue

```

<script setup>
import { computed, onMounted, reactive, ref, watch } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import axios from 'axios'

```

```

import { useCartStore } from '../stores/cartStore.js'
import { t } from '../i18n.js'
import SkySelect from '../components/ui/SkySelect.vue'

const route = useRoute()
const router = useRouter()
const cartStore = useCartStore()

const cart = ref(null)
const items = ref([])
const loading = ref(true)
const busy = ref(false)
const done = ref(false)
const message = ref('')
const confirmState = ref({ open: false, title: '', action: null })
let dismissTimer = null

const confirmHost = computed(() => {
  const host = window.location.hostname || 'localhost'
  const port = window.location.port ? `:${window.location.port}` : ''
  return `${host}${port} says`
})

const emptyFilters = () => ({ keyword: '', sort: 'id' })
const filters = reactive(emptyFilters())
const resetFilters = () => Object.assign(filters, emptyFilters())
const filterActive = computed(() => Object.entries(emptyFilters()).some(([key, value]) =>
filters[key] !== value))
const sortOptions = computed(() => [
  { value: 'id', label: t('filter.sortId') },
  { value: 'nameAsc', label: t('filter.sortNameAsc') },
  { value: 'priceDesc', label: t('filter.sortPriceDesc') },
  { value: 'priceAsc', label: t('filter.sortPriceAsc') },
  { value: 'qtyDesc', label: t('filter.sortQtyDesc') }
])
const sorters = {
  id: (a, b) => String(a.pdId).localeCompare(String(b.pdId), undefined, { numeric: true }),
  nameAsc: (a, b) => String(a.pdName).localeCompare(String(b.pdName)),
  priceAsc: (a, b) => Number(a.price) - Number(b.price),
  priceDesc: (a, b) => Number(b.price) - Number(a.price),
  qtyDesc: (a, b) => Number(b.qty) - Number(a.qty)
}
// filters only change what is listed; the totals below stay over the whole cart
const shownItems = computed(() => {
  const keyword = filters.keyword.toLowerCase()
  const rows = items.value.filter((item) => !keyword
    || [item.pdId, item.pdName].some((field) => String(field ??
'').toLowerCase().includes(keyword)))
  return rows.sort(sorters[filters.sort] || sorters.id)
})

const totalQty = computed(() => items.value.reduce((sum, item) => sum + Number(item.qty), 0))
const totalMoney = computed(() => items.value.reduce((sum, item) => sum + Number(item.price) *
Number(item.qty), 0))

const formattedDate = (value) => {
  if (!value) return '-'
  const date = new Date(value)

```

```

const month = String(date.getMonth() + 1).padStart(2, '0')
const day = String(date.getDate()).padStart(2, '0')
return `${date.getFullYear()}-${month}-${day}`
}

const load = async () => {
  loading.value = true
  try {
    const id = route.params.cartId
    // The API only answers with carts that belong to the signed-in member
    const [master, detail] = await Promise.all([
      axios.get(`http://localhost:3000/carts/getcart/${id}`),
      axios.get(`http://localhost:3000/carts/getcartdtl/${id}`)
    ])
    cart.value = Array.isArray(master.data) ? master.data[0] || null : null
    items.value = Array.isArray(detail.data) ? detail.data : []
  } catch (err) {
    console.log(err.message)
    cart.value = null
  } finally {
    loading.value = false
  }
}

const run = async (action) => {
  busy.value = true
  message.value = ''
  try {
    await action()
    return true
  } catch (err) {
    console.log(err.message)
    done.value = false
    message.value = 'cartdb.actionFail'
    return false
  } finally {
    busy.value = false
  }
}

const setQty = (item, qty) => run(async () => {
  await cartStore.setQty(cart.value.cartId, item.pdId, qty)
  await load()
})

const removeItem = (item) => run(async () => {
  await cartStore.removeProduct(cart.value.cartId, item.pdId)
  await load()
})

const clearSuccessMessage = () => {
  if (dismissTimer) {
    clearTimeout(dismissTimer)
    dismissTimer = null
  }
  message.value = ''
  done.value = false
}

```

```

const closeConfirmDialog = () => {
  confirmState.value = { open: false, title: '', action: null }
}

const runConfirmAction = async () => {
  const action = confirmState.value.action
  closeConfirmDialog()
  if (action) await action()
}

const openConfirmDialog = (title, action) => {
  confirmState.value = {
    open: true,
    title,
    action
  }
}

const removeCart = () => {
  openConfirmDialog(t('cartdb.confirmDelete'), async () => {
    const ok = await run(() => cartStore.deleteCart(cart.value.cartId))
    if (ok) router.push('/cartlist')
  })
}

const confirmOrder = () => {
  openConfirmDialog(t('cartdb.confirmOrder'), async () => {
    const ok = await run(() => cartStore.confirmCart(cart.value.cartId))
    if (!ok) return
    done.value = true
    message.value = 'cartdb.done'
    clearTimeout(dismissTimer)
    dismissTimer = setTimeout(() => {
      clearSuccessMessage()
    }, 2400)
    await load()
  })
}

watch(() => route.params.cartId, load)
onMounted(load)
</script>

```

10.5 CartList.vue (ตะกร้าทั้งหมดที่เคยสั่ง)

src/pages/CartList.vue (ไม่รวม CSS)

vue

```

<template>
  <section class="orders-page sk-page">
    <header class="orders-head sk-head">
      <p class="orders-kicker sk-kicker">{{ t('orders.kicker') }}</p>
      <h1>{{ t('orders.title') }}</h1>
      <p>{{ t('orders.lead') }}</p>
    </header>

    <p v-if="loading" class="orders-state" aria-busy="true">{{ t('db.loading') }}</p>

```

```

<div v-else-if="!authStore.isLogin && cartStore.guestItems.length" class="guest-cart-
layout">
  <div class="guest-cart-items">
    <article v-for="item in cartStore.guestItems" :key="item.pdId" class="guest-cart-
item sk-card">
      
      <div class="guest-cart-item-info">
        <small class="sk-mono">#{{ item.pdId }}</small>
        <h2>{{ item.pdName }}</h2>
        <p class="sk-mono">${{ Number(item.price).toFixed(2) }}</p>
      </div>
      <div class="guest-quantity-control">
        <button type="button" :disabled="item.qty <= 1" :aria-
label="t('cart.decrease', { name: item.pdName })" @click="cartStore.adjustGuestQty(item.pdId,
item.qty - 1)"></button>
        <span class="sk-mono">{{ item.qty }}</span>
        <button type="button" :aria-label="t('cart.increase', { name: item.pdName
})" @click="cartStore.adjustGuestQty(item.pdId, item.qty + 1)"></button>
      </div>
      <strong class="sk-mono">${{ (Number(item.price) *
Number(item.qty)).toFixed(2) }}</strong>
      <button class="guest-remove" type="button" :aria-label="`${t('cart.remove')}${
item.pdName}`" @click="cartStore.removeGuestProduct(item.pdId)"></button>
    </article>
  </div>

  <aside class="guest-checkout sk-panel">
    <p class="orders-kicker sk-kicker">{{ t('cartdb.guestTitle') }}</p>
    <h2>{{ t('cart.summary') }}</h2>
    <div class="guest-summary-line"><span>{{ t('cart.itemsLine', { n: guestQty }) }}
</span><strong class="sk-mono">${{ guestTotal.toFixed(2) }}</strong></div>
    <div class="guest-summary-line"><span>{{ t('cart.shipping') }}</span><strong
class="free">{{ t('cart.free') }}</strong></div>
    <div class="guest-summary-total"><span>{{ t('cart.total') }}</span><strong
class="sk-mono">${{ guestTotal.toFixed(2) }}</strong></div>
    <button class="sk-btn sk-btn-block" type="button" @click="openLoginNotice">{{
t('cartdb.guestPlace') }} <span></span></button>
  </aside>
</div>

<template v-else-if="carts.length">
  <div class="sk-filters" role="group" :aria-label="t('filter.title')">
    <label class="grow">{{ t('filter.cartId') }}<input v-model.trim="filters.keyword"
type="search" :placeholder="t('filter.rowSearchPh')"></label>
    <div class="filter-field grow">
      <span class="filter-label">{{ t('filter.status') }}</span>
      <SkySelect v-model="filters.status" :options="statusOptions" :aria-
label="t('filter.status')" />
    </div>
    <label class="num">{{ t('filter.dateFrom') }}<input v-model="filters.from"
type="date"></label>
    <label class="num">{{ t('filter.dateTo') }}<input v-model="filters.to"
type="date"></label>
    <div class="filter-field grow">
      <span class="filter-label">{{ t('filter.amount') }}</span>
      <SkyRange v-model:min="filters.minAmount" v-model:max="filters.maxAmount"

```

```

:presets="amountPresets" :aria-label="t('filter.amount')" />
    </div>
    <label class="num">{{ t('filter.minQty') }}<input v-model="filters.minQty"
type="number" min="0" step="1" placeholder="0"></label>
    <div class="filter-field grow">
        <span class="filter-label">{{ t('filter.sort') }}</span>
        <SkySelect v-model="filters.sort" :options="sortOptions" :aria-
label="t('filter.sort')" />
    </div>
    <button class="sk-btn sk-btn-ghost" type="button" :disabled="!filterActive"
@click="resetFilters">{{ t('filter.reset') }}</button>
</div>
    <p class="sk-filter-count">{{ t('filter.showing', { n: shown.length, total:
carts.length }) }}</p>

    <div v-if="shown.length" class="orders-table sk-card">
    <table>
        <thead>
            <tr>
                <th>{{ t('orders.no') }}</th>
                <th>{{ t('orders.cartId') }}</th>
                <th>{{ t('orders.date') }}</th>
                <th class="align-center">{{ t('cartdb.qty') }}</th>
                <th class="align-right">{{ t('cartdb.amount') }}</th>
                <th class="align-center">{{ t('orders.status') }}</th>
                <th class="align-center">{{ t('db.actions') }}</th>
            </tr>
        </thead>
        <tbody>
            <tr v-for="cart in shown" :key="cart.cartId">
                <td class="sk-mono">{{ cart.row_number }}</td>
                <td>
                    <router-link :to="`/cartshow/${cart.cartId}`" class="sk-mono order-
link">{{ cart.cartId }}</router-link>
                </td>
                <td>{{ formattedDate(cart.cartDate) }}</td>
                <td class="align-center sk-mono">{{ cart.sqty ?? 0 }}</td>
                <td class="align-right sk-mono">${{ Number(cart.sprice ?? 0).toFixed(2)
}}</td>
                <td class="align-center">
                    <span class="status" :class="cart.cartCf ? 'done' : 'open'">
                        {{ cart.cartCf ? t('cartdb.confirmed') : t('cartdb.open') }}
                    </span>
                </td>
                <td class="align-center">
                    <button class="order-delete" type="button" :aria-
label="`${t('orders.delete')}` ${cart.cartId}`" :disabled="deleting" @click="askDelete(cart)">{{
t('orders.delete') }}</button>
                </td>
            </tr>
        </tbody>
    </table>
</div>
    <div v-else class="sk-empty">
        <span class="sk-empty-icon">◦</span>
        <h2>{{ t('filter.none') }}</h2>
        <p>{{ t('filter.noneText') }}</p>
        <button class="sk-btn" type="button" @click="resetFilters">{{ t('filter.reset')

```

```

    }}</button>
  </div>
</template>

<div v-else class="sk-empty">
  <span class="sk-empty-icon">o</span>
  <h2>{{ t('orders.empty') }}</h2>
  <p>{{ t('orders.emptyText') }}</p>
  <router-link to="/product" class="sk-btn">{{ t('cart.browse') }}</router-link>
</div>

<div v-if="loginNotice" class="notice-overlay" @click.self="loginNotice = false">
  <div class="notice-dialog" role="dialog" aria-modal="true" aria-labelledby="login-notice-title">
    <h2 id="login-notice-title">{{ t('cartdb.loginRequiredTitle') }}</h2>
    <p>{{ t('cartdb.loginRequiredText') }}</p>
    <div class="notice-actions">
      <button class="sk-btn sk-btn-ghost" type="button" @click="loginNotice = false">{{ t('common.close') }}</button>
      <router-link class="sk-btn" to="/login?redirect=/cartlist">{{ t('cartdb.login') }}</router-link>
    </div>
  </div>
</div>

<div v-if="confirmDeleteCart" class="notice-overlay" @click.self="closeDeleteDialog">
  <div class="notice-dialog" role="dialog" aria-modal="true" aria-labelledby="delete-order-title">
    <h2 id="delete-order-title">{{ t('orders.confirmDeleteTitle') }}</h2>
    <p>{{ t('orders.confirmDelete', { id: confirmDeleteCart.cartId }) }}</p>
    <p v-if="deleteError" class="delete-error" role="alert">{{ t('orders.deleteFail') }}</p>
    <div class="notice-actions">
      <button class="sk-btn sk-btn-ghost" type="button" :disabled="deleting" @click="closeDeleteDialog">{{ t('common.close') }}</button>
      <button class="sk-btn sk-btn-coral" type="button" :disabled="deleting" @click="deleteOrder">{{ deleting ? t('db.loading') : t('orders.delete') }}</button>
    </div>
  </div>
</div>
</section>
</template>

<script setup>
import { computed, onMounted, reactive, ref } from 'vue'
import axios from 'axios'
import { t } from '../i18n.js'
import { useAuthStore } from '../stores/authStore.js'
import { useCartStore } from '../stores/cartStore.js'
import SkySelect from '../components/ui/SkySelect.vue'
import SkyRange from '../components/ui/SkyRange.vue'

const carts = ref([])
const loading = ref(true)
const loginNotice = ref(false)
const confirmDeleteCart = ref(null)
const deleting = ref(false)
const deleteError = ref(false)

```

```

const authStore = useAuthStore()
const cartStore = useCartStore()
const guestQty = computed(() => cartStore.guestItems.reduce((sum, item) => sum +
Number(item.qty), 0))
const guestTotal = computed(() => cartStore.guestItems.reduce((sum, item) => sum +
Number(item.price) * Number(item.qty), 0))

const emptyFilters = () => ({ keyword: '', status: '', from: '', to: '', minAmount: '',
maxAmount: '', minQty: '', sort: 'dateDesc' })
const filters = reactive(emptyFilters())
const resetFilters = () => Object.assign(filters, emptyFilters())
const filterActive = computed(() => Object.entries(emptyFilters()).some(([key, value]) =>
filters[key] !== value))

const limit = (value) => value === '' || value === null ? null : (Number.isFinite(Number(value))
? Number(value) : null)
const amountPresets = [{ max: 500 }, { min: 500, max: 2000 }, { min: 2000, max: 10000 }, { min:
10000 }]
const statusOptions = computed(() => [
  { value: '', label: t('filter.all') },
  { value: 'open', label: t('filter.statusOpen') },
  { value: 'done', label: t('filter.statusDone') }
])
const sortOptions = computed(() => [
  { value: 'dateDesc', label: t('filter.sortDateDesc') },
  { value: 'dateAsc', label: t('filter.sortDateAsc') },
  { value: 'amountDesc', label: t('filter.sortAmountDesc') },
  { value: 'amountAsc', label: t('filter.sortAmountAsc') },
  { value: 'qtyDesc', label: t('filter.sortQtyDesc') }
])
const sorters = {
  dateDesc: (a, b) => new Date(b.cartDate) - new Date(a.cartDate),
  dateAsc: (a, b) => new Date(a.cartDate) - new Date(b.cartDate),
  amountDesc: (a, b) => Number(b.sprice ?? 0) - Number(a.sprice ?? 0),
  amountAsc: (a, b) => Number(a.sprice ?? 0) - Number(b.sprice ?? 0),
  qtyDesc: (a, b) => Number(b.sqty ?? 0) - Number(a.sqty ?? 0)
}
const shown = computed(() => {
  const keyword = filters.keyword.toLowerCase()
  const minAmount = limit(filters.minAmount)
  const maxAmount = limit(filters.maxAmount)
  const minQty = limit(filters.minQty)
  const rows = carts.value.filter((cart) => {
    if (keyword && !String(cart.cartId).toLowerCase().includes(keyword)) return false
    if (filters.status === 'open' && cart.cartCf) return false
    if (filters.status === 'done' && !cart.cartCf) return false
    const day = formattedDate(cart.cartDate)
    if (filters.from && day < filters.from) return false
    if (filters.to && day > filters.to) return false
    const amount = Number(cart.sprice ?? 0)
    if (minAmount !== null && amount < minAmount) return false
    if (maxAmount !== null && amount > maxAmount) return false
    if (minQty !== null && Number(cart.sqty ?? 0) < minQty) return false
    return true
  })
  return rows.sort(sorters[filters.sort] || sorters.dateDesc)
})

```

```

const openLoginNotice = () => {
  loginNotice.value = true
}

const askDelete = (cart) => {
  confirmDeleteCart.value = cart
  deleteError.value = false
}

const closeDeleteDialog = () => {
  if (deleting.value) return
  confirmDeleteCart.value = null
  deleteError.value = false
}

const deleteOrder = async () => {
  if (!confirmDeleteCart.value || deleting.value) return
  deleting.value = true
  deleteError.value = false
  try {
    const deletedCartId = confirmDeleteCart.value.cartId
    await
    axios.delete(`http://localhost:3000/database/carts/${encodeURIComponent(deletedCartId)}`)
    carts.value = carts.value
      .filter((cart) => cart.cartId !== deletedCartId)
      .map((cart, index) => ({ ...cart, row_number: index + 1 }))
    confirmDeleteCart.value = null
  } catch (err) {
    console.log(err.message)
    deleteError.value = true
  } finally {
    deleting.value = false
  }
}

const formattedDate = (value) => {
  if (!value) return '-'
  const date = new Date(value)
  const month = String(date.getMonth() + 1).padStart(2, '0')
  const day = String(date.getDate()).padStart(2, '0')
  return `${date.getFullYear()}-${month}-${day}`
}

onMounted(async () => {
  try {
    await cartStore.refresh()
    if (!authStore.isLogin) return
    // POST so the member comes from the token cookie, not the URL
    const res = await axios.post('http://localhost:3000/carts/getcartbycus', {})
    carts.value = Array.isArray(res.data) ? res.data : []
  } catch (err) {
    console.log(err.message)
  } finally {
    loading.value = false
  }
})
</script>

```

11 รับและทดสอบทั้งระบบ

11.1 เปิดโปรแกรม (2 terminal)

Terminal 1 – Backend	bash
<pre>npm run server # API server is running at http://localhost:3000</pre>	
Terminal 2 – Frontend	bash
<pre>npm run dev # Local: http://localhost:5173/</pre>	

เปิดเว็บด้วย <http://localhost:5173> เท่านั้น

cookie ตั้งค่า `sameSite: strict` ถ้าเปิดด้วย `127.0.0.1:5173` browser จะมองว่า `localhost:3000` เป็นคนละเว็บ แล้วไม่ส่ง cookie ทำให้ล็อกอินแล้วหลุดทันที

11.2 รายการทดสอบ

ข้อความในตารางเป็นข้อความเมื่อเลือกภาษา ไทย (ปุ่ม EN / ไทย มุมบนขวา) ค่าเริ่มต้นของเว็บเป็นภาษาอังกฤษ

✓	ทำอะไร	ผลที่ควรได้
☐	เปิด http://localhost:5173/ ตอนยังไม่ล็อกอิน	ถูกพาไป <code>/login</code>
☐	กด "ยังไม่มีบัญชี? ลงทะเบียนใหม่" แล้วสมัคร	ข้อความ "ลงทะเบียนสำเร็จ" และมีแถวใหม่ในตาราง <code>members</code>
☐	ล็อกอินด้วยรหัสผิด / รหัสถูก	"อีเมลหรือรหัสผ่านไม่ถูกต้อง" / ไปหน้า <code>/pagemember</code>
☐	F12 → Application → Cookies	มี <code>token</code> ตี๊ก <code>HttpOnly</code> และ <code>Secure</code>
☐	หน้าสินค้า กด "ดูรายละเอียด"	ไปหน้า <code>/productshow/:pdId</code> เห็นราคายี่ห้อ รายละเอียด
☐	กด "เพิ่มลงตะกร้า"	ปุ่มตะกร้าบนเมนูขึ้นจำนวนและยอดเงิน · ตาราง <code>carts</code> มีแถวใหม่ และ <code>cartDtl</code> มีสินค้า
☐	กดปุ่มตะกร้าบนเมนู	ไปหน้า <code>/cartshow/:cartId</code> เห็นรายการสินค้าและยอดรวม
☐	กดปุ่ม + / - และปุ่ม ×	จำนวนและยอดรวมเปลี่ยนตาม · ลบสินค้าออกจากตะกร้าได้
☐	ใส่สินค้าตัวเดิมซ้ำ	จำนวนบวกเพิ่มในแถวเดิม ไม่เกิดแถวใหม่
☐	กด "ยืนยันสั่งซื้อสินค้า"	ตะกร้าเปลี่ยนสถานะเป็น "สั่งซื้อแล้ว" ปุ่มแก้ไขหายไป และ <code>cartCf</code> ในฐานข้อมูลเป็น <code>true</code>
☐	ใส่สินค้าใหม่หลังยืนยัน	ระบบสร้างตะกร้าใบใหม่ให้

☐	เข้าเมนู "คำสั่งซื้อ"	เห็นตะกร้าทุกใบของตัวเอง เรียงจากล่าสุด
☐	กดออกจากระบบ	ไปหน้า /login , cookie token หายไป

12 ปัญหาที่พบบ่อยและวิธีแก้

อาการ / ข้อความ	สาเหตุ	วิธีแก้
หน้าเว็บขึ้น "เชื่อมต่อเซิร์ฟเวอร์ไม่สำเร็จ" / Server error	PostgreSQL ไม่ได้เปิด หรือค่าใน .env ผิด	เปิด service PostgreSQL ตรวจสอบ DB_* แล้วรีสตาร์ท <code>npm run server</code>
<code>relation "members" does not exist</code> / <code>"carts"</code> / <code>"cartDtl"</code>	ยังไม่ได้สร้างตาราง หรือสร้างผิดฐานข้อมูล	ทำส่วน B ขั้นที่ 3-4 ในฐานข้อมูลที่ตรงกับ DB_NAME
<code>column "cartId" does not exist</code>	สร้างคอลัมน์โดยไม่มี " " จึงกลายเป็นตัวพิมพ์เล็ก	ลบตารางแล้วสร้างใหม่ตาม SQL ในส่วน B
<code>permission denied for table carts</code>	user dev ไม่มีสิทธิ์	รัน GRANT ในหัวข้อ 3.6
กดใส่ตะกร้าแล้วขึ้นว่าต้องเข้าสู่ระบบ	cookie หมดอายุ (1 ชม.) หรือถูกลบ	ล็อกอินใหม่
<code>This cart cannot be changed</code>	ตะกร้าใบนั้นยืนยันสั่งซื้อไปแล้ว	ใส่สินค้าใหม่ ระบบจะสร้างตะกร้าใบใหม่ให้
ปุ่มตะกร้าบนเมนูไม่อัปเดตจำนวน	ยังไม่ได้รีเฟรช store	ปกติจะอัปเดตเอง ถ้าไม่ขึ้นให้รีเฟรชหน้า (F5)
server ไม่ขึ้น: SECRET_KEY is missing	ไม่มี SECRET_KEY ใน .env	เพิ่มค่าตามหัวข้อ 4
Console: blocked by CORS policy	origin ของหน้าเว็บไม่อยู่ใน whitelist	เพิ่ม origin นั้นใน <code>cors({ origin: [...] })</code> ของ <code>server.js</code>
<code>req.cookies</code> เป็น undefined	เรียก <code>cookieParser()</code> หลัง router	ย้าย <code>app.use(cookieParser())</code> ไปก่อน router

D ผังโครงสร้างไฟล์

โปรเจกต์นี้เก็บ backend กับ frontend ไว้ใน repo เดียว ใช้ `package.json` ตัวเดียว แยกกันที่ไฟล์เดอร์และคำสั่งรัน

โครงสร้างไฟล์ทั้งหมด (ไม่รวม node_modules และ dist)

```
WebT/
├─ package.json          สคริปต์: dev (frontend), server (backend), build, preview
├─ vite.config.js        ตั้งค่า Vite
├─ index.html            หน้า host ของ Vue (โหลดฟอนต์, data-theme="sky")
├─ .env                  SECRET_KEY + ค่าเชื่อม PostgreSQL ← ไม่ขึ้น git ต้องสร้างเอง
├─
├─ server/               ===== BACKEND (Express, port 3000) =====
│   ├── index.js          จุดเริ่ม: CORS, cookie-parser, static, ผูก route ทั้งหมด
│   ├── database.js       pg Pool อ่านค่าจาก .env
│   ├── controllers/
│   │   ├── authMiddleware.js requireLogin: ตรวจ JWT ใน cookie แล้วใส่ req.member
│   │   ├── memberController.js สมักร / login / logout / แก้โปรไฟล์ / อัปโหลด-ลบรูป (Lab 09, 10, 12)
│   │   ├── productController.js CRUD สินค้า + รูปสินค้า + ค้นหา
│   │   ├── cartController.js   ตกร้าทั้งหมดตาม Lab 11
│   │   └── databaseController.js ดูทุกตาราง + ลบ member / cart / แถวตารางอ้างอิง
│   ├── swagger.yaml      OpenAPI spec ที่ Swagger UI อ่าน (/api-docs)
│   └─ routes/
│       ├── memberRoute.js    /members/*
│       ├── productRoute.js   /products/*, /search/products/:id
│       ├── cartRoute.js      /carts/*
│       └── databaseRoute.js  /database/*
├─ src/                  ===== FRONTEND (Vue 3 + Vite, port 5173) =====
│   ├── main.js           createApp + Pinia + Router + axios.withCredentials
│   ├── App.vue           MainMenu + router-view
│   ├── i18n.js           ค่าแปล EN/TH ทุกข้อความ + แปลข้อความที่ API ส่งมา
│   ├── router/index.js   เส้นทางหน้าเว็บ + guard requiresAuth / guestOnly
│   ├── stores/
│   │   ├── authStore.js   สถานะ login และข้อมูลสมาชิก
│   │   └── cartStore.js   ตกร้าในฐานข้อมูล + ตกร้า guest ใน localStorage
│   ├── pages/            หนึ่งไฟล์ = หนึ่งหน้าใน router
│   │   ├── TheHome.vue    หน้าแรก
│   │   ├── PageProduct.vue รายการสินค้า + ค้นหา + ฟิลเดอร์
│   │   ├── ProductShow.vue รายละเอียดสินค้า
│   │   ├── ProductManage.vue เพิ่ม / แก้ / ลบสินค้า + อัปโหลดรูปสินค้า
│   │   ├── CartShow.vue   ตกร้าใบเดียว (แก้จำนวน ลบ ยืนยันสั่งซื้อ)
│   │   ├── CartList.vue   ประวัติคำสั่งซื้อ + ตกร้าก่อนล็อกอิน
│   │   ├── DatabaseOverview.vue ดูทุกตาราง + ลบข้อมูล
│   │   ├── PageMember.vue โปรไฟล์ + รูปสมาชิก (Lab 12)
│   │   ├── TheLogin.vue   เข้าสู่ระบบ
│   │   └── TheRegister.vue สมักรสมาชิก
│   ├── components/
│   │   ├── layout/        MainMenu, AuthLayout, CartInfo, AppSwitches
│   │   └── ui/            ProductCard, SkySelect, SkyRange, SegmentSwitch, SkyIcon, DevBypass
│   └─ assets/            base.css, main.css, skylearn.css (ดีไซน์หลัก + คลาส .sk-* ที่ใช้ร่วมกัน)
├─ img_pd/               รูปสินค้า (backend เสิร์ฟที่ /img_pd)
├─ img_mem/              รูปสมาชิก: default.jpg + avatar-1..3.jpg
│                           (รูปที่สมาชิกอัปโหลดถูก .gitignore ไว้)
└─ public/favicon.ico
```

```

├─ dist/                  ผลลัพธ์ npm run build → เอาไป copy ลง NGINX
├─ docs/
│  └─ Lab09-12-Guide.pdf  เอกสารฉบับนี้
│  └─ build-guide.mjs     สคริปต์สร้าง PDF ฉบับนี้ (node docs/build-guide.mjs)
│  └─ nginx.conf.example  ตัวอย่าง config สำหรับ deploy

```

ง่าย ๆ

โค้ดที่รันบนเซิร์ฟเวอร์อยู่ใน `server/` ทั้งหมด โค้ดที่รันในเบราว์เซอร์อยู่ใน `src/` ทั้งหมด ไฟล์รูปและ `.env` อยู่ที่ราก

อยากทำอะไร	ไปที่ไฟล์
เพิ่ม API ใหม่	<code>server/controllers/...</code> เขียนฟังก์ชัน แล้วผูกใน <code>server/routes/...</code>
เพิ่มหน้าใหม่	สร้างไฟล์ใน <code>src/pages/</code> แล้วเพิ่ม route ใน <code>src/router/index.js</code>
แก้ข้อความบนหน้าเว็บ	<code>src/i18n.js</code> (ต้องแก้ทั้งบล็อก en และ th)
แก้สี ฟอนต์ ปุ่ม	<code>src/assets/skylearn.css</code>
แก้เมนูด้านบน	<code>src/components/layout/MainMenu.vue</code>

13 Lab 12: อัปโหลดและแสดงรูปสมาชิก

รูปสมาชิกเก็บเป็นไฟล์ในโฟลเดอร์ `img_mem` ไม่ได้เก็บลงฐานข้อมูล ชื่อไฟล์คืออีเมลของสมาชิก เช่น `a@ku.th.jpg` ใครยังไม่มีรูปจะใช้ `default.jpg`

Backend

1. สร้างโฟลเดอร์ `img_mem` ที่รากโปรเจกต์ แล้วใส่ `default.jpg`
2. เปิดเป็น static path ใน `server/index.js`
3. ติดตั้ง `multer` (`npm install multer`) แล้วเขียนตัวรับไฟล์

server/index.js – เปิดโฟลเดอร์รูปให้เข้าถึงได้

js

```
app.use("/img_pd", express.static("img_pd"));
app.use("/img_mem", express.static("img_mem"));
```

server/controllers/memberController.js – ตั้งค่า multer

js

```
// The photo is named after the signed-in member, so strip anything that could walk the path
const reserved = /^(default|avatar-.*$/i;
function photoStem(memEmail) {
  const stem = String(memEmail).replace(/^[A-Za-z0-9@._-]/g, '_');
  // never let a member claim default.jpg or one of the shipped avatars
  return reserved.test(stem) ? '_' + stem : stem;
}
const photoDir = 'img_mem';
const photoExt = (mimetype) => (mimetype === 'image/png' ? 'png' : mimetype === 'image/webp' ? 'webp' : 'jpg');

// the photo keeps its real extension, so look the file up by its stem
function findPhoto(memEmail) {
  const stem = photoStem(memEmail);
  const found = fs.readdirSync(photoDir).find((name) => path.parse(name).name === stem);
  return found ? `/${photoDir}/${found}` : null;
}

// a fresh clone may not have the folder yet, and multer will not create it
fs.mkdirSync("img_mem", { recursive: true });

const photoUpload = multer({
  storage: multer.diskStorage({
    destination: (req, file, callback) => callback(null, 'img_mem'),
    // the owner comes from the token, never from the form body
    filename: (req, file, callback) => callback(null,
`${photoStem(req.member.memEmail)}.${photoExt(file.mimetype)}`),
  }),
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (req, file, callback) => {
    callback(null, ['image/jpeg', 'image/png', 'image/webp'].includes(file.mimetype));
  }
}).single('file');

// requireLogin runs first, so req.member is already the verified token
```

```

export async function uploadMemberPhoto(req, res) {
  console.log(`POST /members/uploading is requested.`);
  photoUpload(req, res, (err) => {
    if (err) {
      console.log(err.message);
      return res.status(400).json({ error: 'photo.uploadFail' });
    }
    if (!req.file) {
      return res.status(400).json({ error: 'photo.invalid' });
    }
    // a member has one photo: drop the copies left over in other formats
    const stem = photoStem(req.member.memEmail);
    for (const name of fs.readdirSync(photoDir)) {
      if (path.parse(name).name === stem && name !== req.file.filename) {
        fs.unlinkSync(path.join(photoDir, name));
      }
    }
    return res.json({ message: 'photo.uploaded', photo: `/${photoDir}/${req.file.filename}`
  });
});
}

```

server/controllers/memberController.js – ตัวรับไฟล์และตัวลบรูป

js

```

export async function uploadMemberPhoto(req, res) {
  console.log(`POST /members/uploading is requested.`);
  photoUpload(req, res, (err) => {
    if (err) {
      console.log(err.message);
      return res.status(400).json({ error: 'photo.uploadFail' });
    }
    if (!req.file) {
      return res.status(400).json({ error: 'photo.invalid' });
    }
    // a member has one photo: drop the copies left over in other formats
    const stem = photoStem(req.member.memEmail);
    for (const name of fs.readdirSync(photoDir)) {
      if (path.parse(name).name === stem && name !== req.file.filename) {
        fs.unlinkSync(path.join(photoDir, name));
      }
    }
    return res.json({ message: 'photo.uploaded', photo: `/${photoDir}/${req.file.filename}`
  });
});

export async function deleteMemberPhoto(req, res) {
  console.log(`DELETE /members/photo is requested.`);
  try {
    const stem = photoStem(req.member.memEmail);
    let removed = false;
    for (const name of fs.readdirSync(photoDir)) {
      if (path.parse(name).name === stem) {
        fs.unlinkSync(path.join(photoDir, name));
        removed = true;
      }
    }
    if (!removed) return res.status(404).json({ error: 'photo.none' });
  }
}

```

```

    return res.json({ message: 'photo.removed', photo: null });
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: 'photo.removeFail' });
  }
}

```

server/routes/memberRoute.js – สองเส้นทางที่เพิ่ม

js

```

router.post("/members/uploading", requireLogin, memberC.uploadMemberPhoto);
router.delete("/members/photo", requireLogin, memberC.deleteMemberPhoto);

```

ต่างจากเอกสารแลบตรงนี้

แลบตั้งชื่อไฟล์จาก `req.body.memEmail` ที่ส่งมาที่ฟอร์ม ใครก็แก้ค่านั้นเป็นอีเมลคนอื่นแล้วทำรูปคนอื่นได้ หรือใส่ `../..` เพื่อเขียนไฟล์นอกโฟลเดอร์ในโปรเจกต์นี้ชื่อไฟล์มาจาก `token ( req.member.memEmail )` กรองตัวอักษรที่ใช้ไม่ได้ทั้ง จำกัดขนาด 5 MB และรับเฉพาะ jpeg / png / webp

Frontend

หน้า `src/pages/PageMember.vue` แสดงรูป มีฟอร์มอัปโหลด ปุ่มลบรูป และอวตารสำเร็จรูปให้เลือก

PageMember.vue – ส่งไฟล์ด้วย FormData

js

```

const sendPhoto = async (blob, name) => {
  uploading.value = true
  fileMessage.value = ''
  fileError.value = false
  try {
    const formData = new FormData()
    formData.append('file', blob, name)
    const response = await axios.post(`${API}/members/uploading`, formData, {
      headers: { 'Content-Type': 'multipart/form-data' }
    })
    fileMessage.value = response.data.message || 'photo.uploaded'
    photoPath.value = response.data.photo || photoPath.value
    photoStamp.value = Date.now()
  } catch (err) {
    fileError.value = true
    fileMessage.value = err.response?.data?.error || 'photo.uploadFail'
  } finally {
    uploading.value = false
  }
}

```

PageMember.vue – URL ของรูปและการกันแคช

js

```

const photoStamp = ref(Date.now())
const photoUrl = computed(() => photoPath.value
  ? `${API}${photoPath.value}?t=${photoStamp.value}`
  : null)
photoPath.value = member.value?.photo || null
photoStamp.value = Date.now()
photoStamp.value = Date.now()

```

ทำไมต้องมี ?t=

ถ้าอัปโหลดรูปใหม่ทับชื่อไฟล์เดิม เบราวเซอร์จะยังโชว์รูปเก่าจากแคช จึงต่อท้าย URL ด้วยเวลาปัจจุบัน (?t=1735...) ทุกครั้งที่อัปโหลดสำเร็จ URL เปลี่ยน เบราวเซอร์จึงโหลดรูปใหม่

ต่างจากแลบ: ไม่ต้องเดานามสกุลไฟล์

แลบใช้ `new Image()` ลองโหลด `<email>.jpg` เพื่อดูว่ามีรูปไหม โปรเจกต์นี้เก็บนามสกุลจริง (png อัปโหลดมาก็เก็บเป็น .png) แล้วให้ `GET /members/detail` ตอบ `photo` เป็น path ของไฟล์จริงมาเลย หน้าเว็บจึงไม่ต้องเดา

14 Lab 12: Deploy ด้วย NGINX

1. โหลด NGINX for Windows จาก nginx.org/en/download.html แยกไฟล์ไว้โฟลเดอร์ที่ต้องการ
2. ที่โปรเจกต์สั่ง `npm run build` จะได้โฟลเดอร์ `dist/`
3. copy ทุกไฟล์ใน `dist/` ไปไว้ใน `<nginx>/html/`
4. แก้ `<nginx>/conf/nginx.conf` ตามตัวอย่างด้านล่าง แล้วตรวจไวยากรณ์ด้วย `nginx -t`
5. สั่ง `start nginx` แล้วเปิด `http://localhost` (ต้องรัน backend `npm run server` ไว้ด้วย)

nginx.conf (ส่วน server)

nginx

```
server {
    listen      80;
    server_name localhost;

    root    html;
    index  index.html;

    # Vue Router ใช้ history mode: ถ้าไม่มีไฟล์จริงให้ส่ง index.html กลับไป
    # ไม่งั้นกด refresh ที่ /product หรือ /cartlist แล้วจะขึ้น 404
    location / {
        try_files $uri $uri/ /index.html;
    }

    # ไฟล์ static ของ Vite มี hash ในชื่ออยู่แล้ว cache ได้ยาว
    location /assets/ {
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root    html;
    }
}
```

ห้ามลืม try_files

Vue Router ใช้ history mode ถ้าไม่ใส่ `try_files $uri $uri/ /index.html;` พอเปิด `localhost/cartlist` ตรง ๆ หรือกด refresh จะได้ 404 เพราะ NGINX หาไฟล์ชื่อ `cartlist` ไม่เจอ

server/index.js – CORS ต้องรับ origin ของ NGINX และ preview ด้วย

js

```
app.use(cors({
  origin: [
    "http://localhost:5173", "http://127.0.0.1:5173",
    // vite preview and the nginx build (port 80)
    "http://localhost:4173", "http://127.0.0.1:4173",
    "http://localhost", "http://127.0.0.1"
  ],
  methods: ["GET", "POST", "PUT", "DELETE"],
```

```
credentials: true
});
```

คำสั่ง	ความหมาย
<code>start nginx</code>	เริ่มทำงานแบบ background
<code>nginx -s stop</code>	หยุดทันที
<code>nginx -s reload</code>	โหลด config ใหม่โดยไม่หยุดบริการ
<code>nginx -t</code>	ตรวจไวยากรณ์ config ก่อนใช้จริง

พอร์ต 80 ขนกับโปรแกรมอื่น

ถ้า `start nginx` แล้วเปิด localhost ไม่ขึ้น ให้ดูใน `<nginx>/logs/error.log` ส่วนใหญ่เกิดจากมีโปรแกรมอื่นจองพอร์ต 80 อยู่ แก้โดยเปลี่ยน `listen 80;` เป็นพอร์ตอื่น เช่น `8080` แล้วเพิ่ม origin นั้นใน CORS ด้วย

15 Lab 13: เอกสาร API ด้วย Swagger

Swagger UI อ่านไฟล์ OpenAPI (เขียนด้วย YAML) แล้วสร้างหน้าเอกสาร API ที่กดทดลองยิง request ได้ทันที เปิดดูได้ที่ <http://localhost:3000/api-docs>

ติดตั้ง

Terminal

bash

```
npm install swagger-ui-express yaml
```

ผูกเข้ากับ Express

ใส่ไว้หลัง การผูก route ทั้งหมดใน `server/index.js`

server/index.js

js

```
// API document (Lab 13): the spec lives next to this file, so it is found
// no matter which folder the server was started from
const here = path.dirname(fileURLToPath(import.meta.url));
const swaggerFile = fs.readFileSync(path.join(here, "swagger.yaml"), "utf-8");
const swaggerDoc = yaml.parse(swaggerFile);
app.use("/api-docs", swaggerUI.serve, swaggerUI.setup(swaggerDoc));
```

ทำไมต้องใช้ `path.dirname(fileURLToPath(import.meta.url))`

เอกสารแลปอ่านไฟล์ด้วย path ตรง ๆ (`services/swagger.yaml`) ซึ่งอิงกับโฟลเดอร์ที่สั่งรัน ถ้ารันจากที่อื่นจะหาไฟล์ไม่เจอ ในโปรเจกต์นี้อ้างอิงจากตำแหน่งของ `server/index.js` เอง จึงรันจากที่ไหนก็ได้

โครงของไฟล์ YAML

- **Comment** ขึ้นต้นด้วย `#`
- **Key-value** คู่กันด้วย `:` และหนึ่งช่องว่าง
- **Indentation** ใช้ช่องว่าง (ห้าม tab) การเยื้องบอกว่าอะไรเป็นลูกของอะไร เหมือน Python
- **List** ขึ้นต้นแต่ละรายการด้วย `-`
- `|` หลัง key แปลว่าค่าถัดไปเป็นข้อความหลายบรรทัด

server/swagger.yaml – ส่วนหัวและ server

yaml

```
openapi: 3.0.4

info:
  title: KUSHOP API
  description: |
    KUSHOP Project from Kasetsart University Sriracha Campus.<br>
    เอกสารนี้อธิบายทุก endpoint ของ backend: สินค้า สมาชิก ตะกร้าสินค้า และการดูแลข้อมูล<br><br>
    <b>การยืนยันตัวตน</b> ใช้ JWT ที่เก็บใน cookie ชื่อ token แบบ httpOnly
    ซึ่งเซิร์ฟเวอร์ตั้งให้ตอน POST /members/login สำเร็จ
    เบราวเซอร์จะแนบ cookie ให้เองถ้าเรียกจาก origin ที่อยู่ใน CORS whitelist
```

(5173, 4173 และ port 80)

version: "1.0"

servers:

- url: http://localhost:3000/
description: Local development

GET: อธิบาย endpoint และ response

server/swagger.yaml – GET /products

yaml

```
/products:
  get:
    tags: [Products]
    summary: Returns a list all of products.
    description: |
      List all products in array of object format.<br>
      Each product contains details like product ID, name, price, brand and product type.
    responses:
      '200':
        description: A JSON array of Products
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/products'
      '500':
        description: Server error.

  post:
    tags: [Products]
    summary: Creates a new Product.
    description: เฉพาะแอดมิน รหัสสินค้าจะถูกสร้างให้อัตโนมัติจากเลขล่าสุด
    security:
      - cookieAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/new_product'
    responses:
      '201':
        description: Product is created.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/products'
      '400':
        description: Required fields are missing or the price is invalid.
      '401':
        description: Login is required.
      '403':
        description: Admin role is required.
      '409':
        description: Product ID is existed.
```

```
'500':  
  description: Server Error
```

`$ref` คือการอ้างอิงไปที่ schema ที่ประกาศไว้ใน `components` จะได้ไม่ต้องเขียนรายละเอียดซ้ำทุกที่

GET + Parameters

endpoint ที่มีตัวแปรใน path เขียนชื่อไว้ในวงเล็บปีกกา แล้วประกาศไว้ได้ `parameters`

server/swagger.yaml – GET /products/{id}

yaml

```
/products/{id}:  
  get:  
    tags: [Products]  
    summary: Returns the specify products.  
    description: Show a product in object format.  
    parameters:  
      - name: id  
        in: path  
        required: true  
        description: Product ID  
        schema:  
          type: string  
          example: "001"  
    responses:  
      '200':  
        description: A JSON object of the Product  
        content:  
          application/json:  
            schema:  
              $ref: '#/components/schemas/products'  
      '404':  
        description: A product with the specified ID was not found.  
      '500':  
        description: Server error.
```

POST ผ่าน Body

endpoint เดียวกันแต่คนละ method ให้เขียนไว้ได้ path เดียวกัน แยกกันที่ชื่อ method

server/swagger.yaml – POST /products และ schema ที่ใช้

yaml

```
post:  
  tags: [Products]  
  summary: Creates a new Product.  
  description: เฉพาะแอดมิน รหัสสินค้าจะถูกสร้างให้อัตโนมัติจากเลขล่าสุด  
  security:  
    - cookieAuth: []  
  requestBody:  
    required: true  
    content:  
      application/json:  
        schema:  
          $ref: '#/components/schemas/new_product'  
  responses:  
    '201':
```

```

    description: Product is created.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/products'
  '400':
    description: Required fields are missing or the price is invalid.
  '401':
    description: Login is required.
  '403':
    description: Admin role is required.
  '409':
    description: Product ID is existed.
  '500':
    description: Server Error

new_product:
  type: object
  required: [pdName, pdPrice, brandId, brandName, pdTypeId]
  properties:
    pdName:
      type: string
    pdPrice:
      type: number
    pdRemark:
      type: string
    pdTypeId:
      type: string
    brandId:
      type: string
    brandName:
      type: string

```

บอกว่า endpoint ไหนต้องล็อกอิน

ประกาศวิธียืนยันตัวตนไว้ที่ `components.securitySchemes` แล้วใส่ `security` ให้ endpoint ที่ต้องใช้ Swagger UI จะขึ้นรูปกุญแจที่ endpoint นั้น

server/swagger.yaml – cookie auth

yaml

ทดลองยิงจากหน้า Swagger

cookie เป็น httpOnly + SameSite=Strict ปุ่ม Try it out จะใช้ได้ก็ต่อเมื่อ login ผ่านหน้าเว็บในเบราว์เซอร์เดียวกันมาก่อน เพราะ Swagger UI ยิงจาก origin `localhost:3000` ซึ่งเบราว์เซอร์จะแนบ cookie ให้เอง ส่วน endpoint ที่ต้องเป็นแอดมินก็ต้องล็อกอินด้วยบัญชีแอดมิน

สิ่งที่เอกสารนี้ครอบคลุม

กลุ่ม	จำนวน	ตัวอย่าง endpoint
Products	9	GET /products , POST /products , DELETE /products/{id}
Members	8	POST /members/login , POST /members/uploading
Carts	11	POST /carts/addcartdtl , PUT /carts/cfcart/{id}
Database	5	GET /database/overview , PUT /database/members/{memEmail}/role

เอกสารต้องตรงกับโค้ดเสมอ

ทุก route ที่ Express ผู้ถูกเขียนไว้ในไฟล์ YAML ครบทั้ง 33 รายการ เวลาเพิ่ม route ใหม่ให้เพิ่มใน `server/swagger.yaml` ด้วย ไม่งั้นเอกสารจะบอกไม่ตรงกับของจริง

16 ส่วนที่ทำเพิ่มนอกแลป

ส่วนเหล่านี้ไม่มีในเอกสารแลป แต่อยู่ในโปรเจกต์แล้ว

เรื่อง	รายละเอียด
ฟิลเตอร์ทุกหน้า	หน้าสินค้า (แบรนด์ ประเภท ช่วงราคา การเรียง) หน้าจัดการสินค้า (คำค้น แบรนด์ ประเภท ช่วงราคา) ประวัติคำสั่งซื้อ (รหัสตระกร้า สถานะ ช่วงวันที่ ช่วงยอดเงิน จำนวน) ตะกร้า (คำค้น การเรียง) และหน้าฐานข้อมูล (กรองชื่อตาราง ค้นหาในแถว เลือกคอลัมน์) ทั้งหมดกรองฝั่งเบราร์เซอร์จากข้อมูลที่โหลดมาแล้ว ไม่ยิง API เพิ่ม
ตัวเลือกแบบ custom	<code>src/components/ui/SkySelect.vue</code> แทน <code><select></code> ทุกจุด รองรับคีย์บอร์ดครบ และ <code>SkyRange.vue</code> สำหรับช่วงราคา / ยอดเงิน เลือกจากช่วงสำเร็จรูปหรือกรอกเองก็ได้
ลบข้อมูลได้ทุกส่วน	สินค้า (DELETE /products/:id ลบไฟล์รูปตามไปด้วย) แถวของ products / brands / pdTypes ในหน้าฐานข้อมูล (DELETE /database/rows/:table/:id เฉพาะ admin และชื่อตารางมาจาก whitelist ในโค้ด) สมาชิก ตะกร้า สินค้าในตะกร้า และรูปโปรไฟล์
สองภาษา	ทุกข้อความอยู่ใน <code>src/i18n.js</code> ทั้ง en และ th รวมถึงข้อความที่ API ส่งกลับมา (API ส่งเป็นคีย์ เช่น <code>photo.uploaded</code> แล้วหน้าเว็บแปลเอง)

17 สิ่งต่างจากเอกสารแลป

เอกสารแลป	โปรเจกต์นี้	เหตุผล
bcrypt	bcryptjs	มีในโปรเจกต์แล้ว API เหมือนกัน ไม่ต้องคอมไพล์บน Windows
ไฟล์ index.js , routes/ , services/database.js	server/index.js , server/routes/ , server/database.js	แยก backend ทั้งหมดไว้ในโฟลเดอร์ server/
ตัวแปร DBSERVER , DBUSER , ...	DB_HOST , DB_USER , ...	ชื่อที่ database.js ใช้อยู่
postMember ส่ง bodyData กลับทั้งก่อน	ส่งกลับแค่อีเมล ชื่อ วันที่ และผล	ไม่ให้รหัสผ่านถูกส่งกลับไปใน response
SECRET_KEY='thailandgogo'	ค่าสุ่ม 64 ตัวอักษร และ server ไม่ยอมเปิดถ้าไม่มี	เดา key ไม่ได้ = ปลอม token ไม่ได้
แต่ละหน้าเรียก /members/detail เอง และ MainMenu ใช้ watch	authStore.getMember() ที่เดียว + router guard	ทุกหน้าเห็นข้อมูลชุดเดียวกัน ไม่ยิง API ซ้ำ
ตะกร้าส่ง memEmail / cusId มาใน body	อ่านจาก token ใน cookie	ค่าใน body ปลอมเป็นคนอื่นได้ทั้งการดูและการแก้ตะกร้า
postCartDtl รับราคาจาก req.body.pdPrice	อ่านราคาจากตาราง products	ถ้าเชิ้อราคาจากหน้าเว็บ ผู้ใช้แก้เป็น 0.01 บาทแล้วสั่งซื้อได้
ไม่ได้ตรวจสอบสถานะตะกร้าตอนแก้ไข	ตะกร้าที่ยืนยันแล้วแก้ไม่ได้ (canEditCart)	คำสั่งซื้อที่ยืนยันแล้วไม่ควรเปลี่ยนยอดได้อีก
ปุ่มลบตะกร้า / ยืนยันสั่งสินค้า มีแต่ confirm()	ต่อกับ delCart และ cfCart จริง	ให้ปุ่มทำงานได้จริง
ตะกร้าเก็บใน localStorage (ของเดิมในโปรเจกต์)	เก็บในฐานข้อมูลตาม Lab 11	ตะกร้าผูกกับสมาชิก ใช้ข้ามเครื่องได้และมีประวัติคำสั่งซื้อ
ฟอร์ม Bootstrap ตามแลป	ดีไซน์ใหม่ + สลับภาษา ไทย/อังกฤษ	ส่วนที่ออกแบบเพิ่มในโปรเจกต์ (ถอด Bootstrap ออกแล้ว)
Lab 12: ชื่อไฟล์รูปจาก req.body.memEmail	ชื่อไฟล์จาก token และกรองตัวอักษรก่อนใช้	กันการทับรูปคนอื่นและการเขียนไฟล์นอกโฟลเดอร์
Lab 13: อ่าน services/swagger.yaml ด้วย path ตรง ๆ และอธิบายเฉพาะ /products	อ่านไฟล์จากตำแหน่งของ server/index.js และอธิบายครบทั้ง 33 endpoint	รันจากโฟลเดอร์ไหนก็หาไฟล์เจอ และเอกสารตรงกับ API จริงทั้งหมด
Lab 12: บันทึกเป็น .jpg เสมอ และเดาว่ามีรูปใหม่ด้วย new Image()	เก็บนามสกุลจริง และ GET /members/detail บอก path ของรูป	ไฟล์ png ไม่ถูกตั้งชื่อผิดชนิด และหน้าเว็บไม่ต้องเดา

	มาเลย	
--	-------	--